

TripCraft — 설계 문서

관광지 탐색 · 드래그앤드롭 일정 확정 · 이동시간 자동 계산 · 커뮤니티 공유

개발 · 전진 · 송정기

목차

1. 요구사항 정의서
2. Use-Case 다이어그램
3. 클래스 다이어그램
4. ER 다이어그램
5. WBS & 간트 차트
6. 화면 설계서
7. 기타 참고 문서 (기술 스택 · 시스템 구조 · 외부 API · 컨벤션 · 알고리즘)
8. API 명세
9. 기술 심화 ① — 경로 최적화 & 지도 시각화
10. 기술 심화 ② — 실시간 협업 & 동시성

요구사항 정의서

본 문서의 §1~§8은 기획 단계 요구사항(Phase 1 MVP + AI 챗봇)이다. MVP 안정화 이후 추가 구현된 기능(실시간 협업·공유 링크, 마이페이지, 댓글·북마크, 방문 지도)은 §9 구현 확장 요구사항에 정리했다.

목차

- 문서 개요
- 사용자 역할 (Actor) 정의
- 핵심 사용자 플로우
- 기능 요구사항 — Phase 1 (MVP)
- 기능 요구사항 — Phase 2 (고도화)
- 비기능 요구사항
- 제약 사항 및 가정
- 기능 우선순위 최종 확인

1. 문서 개요

목적

본 문서는 TripCraft 프로젝트의 기능 요구사항을 사용자 스토리(User Story)와 상세 명세 형식으로 정의한다. 개발팀(백엔드·프론트엔드), 설계(ERD·API 명세), 테스트(QA) 작업의 기준 문서로 활용한다.

표기 규칙

- [필수]:** 해당 기능이 없으면 MVP 완성 불가. 일정 내 반드시 구현.
- [권장]:** MVP 범위이나 일정 압박 시 범위 축소 가능. 별도 표기.
- [Phase 2]:** MVP 범위 외. 안정화 후 다음 스프린트에서 구현.
- 수용 기준(Acceptance Criteria, AC)은 QA·테스트 기준으로 활용한다.

2. 사용자 역할 (Actor) 정의

역할	설명	주요 권한
비회원 (Guest)	로그인하지 않은 방문자	관광지 조회, 커뮤니티 열람, 공지사항 열람
회원 (Member)	로그인한 일반 사용자	비회원 권한 + 일정 생성·편집·저장, 즐겨찾기, 게시물 작성
관리자 (Admin)	서비스 운영자	회원 권한 + 공지사항 CRUD, 게시물 관리

3. 핵심 사용자 플로우

서비스의 핵심 시나리오를 플로우 형태로 기술한다. 세부 화면 명세는 Figma 명세서(docs/02_design/wireframe_spec.md) 참조.

- Step 1. 일정 생성
 - 회원이 GNB "내 일정" 탭 클릭 또는 "+ 새 일정 만들기" 선택
 - 일정 생성 모달에서 여행 제목·출발일·귀환일·인원 입력
 - "만들기" 클릭 → 일정 생성, 화면 B(내 일정)로 이동
- Step 2. 후보군 등록 (화면 A - 관광지 탐색)
 - 지역 / 카테고리 필터로 관광지 탐색
 - 원하는 장소 카드에서 "일정에 추가 +" 클릭
 - 선택 장소가 활성 일정의 후보군에 등록됨 (도시별 자동 분류)
 - 해당 도시 첫 등록 시: 즐겨찾기 동일 도시 장소가 후보군에 자동 추가 + 토스트 알림
- Step 3. 일정 확정 (화면 B - 내 일정)
 - 좌측 후보군 사이드바(도시 그룹별 CandidateCard) 확인
 - CandidateCard를 우측 타임라인 DropZone으로 드래그 앤 드롭
 - PlaceBlock 생성 → ODsay API로 이전·이후 장소와의 이동 시간 자동 계산·삽입
 - 블록 재배치 / 삭제 / 시간 수정 → 이동 시간 재계산
- Step 4. 일정 공유 (화면 C - 커뮤니티)
 - 완성된 일정을 커뮤니티 게시판에 공유
 - 다른 사용자의 공유 일정 열람 및 참고

4. 기능 요구사항 — Phase 1 (MVP)

F12 — 회원 관리

가장 먼저 구현해야 하는 기반 기능. 이후 모든 개인화 기능의 전제 조건.

사용자 스토리

ID	역할	행동	목적
US-12-01	방문자	이메일·닉네임·비밀번호로 회원가입을 한다	서비스의 개인화 기능을 이용하기 위해
US-12-02	방문자	이메일·비밀번호로 로그인한다	내 일정과 즐겨찾기에 접근하기 위해
US-12-03	회원	로그아웃한다	공용 기기에서 내 계정을 보호하기 위해
US-12-04	회원	닉네임·비밀번호를 수정한다	최신 정보로 계정을 유지하기 위해
US-12-05	회원	계정을 탈퇴한다	서비스에 저장된 내 데이터를 삭제하기 위해

기능 명세

회원가입 [필수]

- 입력 항목: 이메일, 닉네임, 비밀번호, 비밀번호 확인
- 이메일: 유효한 형식(xxx@xxx.xxx), 중복 불가
- 닉네임: 2~20자, 특수문자 제한
- 비밀번호: 8자 이상, 영문·숫자 포함 필수
- 비밀번호는 BCrypt로 해시하여 저장

- 회원가입 후 자동 로그인 처리

로그인 / 로그아웃 [필수]

- 로그인 성공 시 Access Token (30분) + Refresh Token (7일) 발급
- Access Token·Refresh Token 모두 **HttpOnly 쿠키**로 발급·전달 (access_token · refresh_token)
- 보호된 요청은 서버가 access_token 쿠키를 검증(JwtAuthenticationFilter)
- 로그아웃 시 Refresh Token 무효화

회원 정보 수정 [필수]

- 수정 가능 항목: 닉네임, 비밀번호
- 비밀번호 변경 시 현재 비밀번호 재확인

회원 탈퇴 [필수]

- 비밀번호 재확인 후 탈퇴 처리
- 탈퇴 시 일정·즐거찾기·좋아오는 삭제. 작성한 게시물·공지는 보존 ("탈퇴한 사용자"로 표시)

수용 기준 (Acceptance Criteria)

- AC-12-01: 이미 사용 중인 이메일로 가입 시 "이미 사용 중인 이메일입니다." 오류 메시지 노출
- AC-12-02: 비밀번호 확인 불일치 시 가입 버튼 비활성화
- AC-12-03: 잘못된 이메일 또는 비밀번호 입력 시 로그인 실패 메시지 노출, 계정 정보 노출 없음
- AC-12-04: Access Token 만료 시 Refresh Token으로 자동 재발급됨
- AC-12-05: 로그인하지 않은 상태에서 보호된 페이지 접근 시 로그인 페이지로 리다이렉트
- AC-12-06: 탈퇴 후 해당 계정으로 로그인 불가

F01 · F02 · F03 — 관광지 DB 구축 및 조회

사용자 스토리

ID	역할	행동	목적
US-01-01	시스템	한국관광공사 API에서 관광지 데이터를 수집하여 DB에 저장한다	사용자가 외부 사이트 이탈 없이 관광지를 탐색할 수 있도록
US-02-01	비회원/회원	시도·시군구 단위로 관광지를 필터링하여 조회한다	여행 목적지의 장소만 모아서 보기 위해
US-03-01	비회원/회원	관광지·음식점·숙박·문화시설 등 카테고리별로 조회한다	원하는 종류의 장소를 빠르게 찾기 위해
US-03-02	비회원/회원	관광지명으로 검색한다	특정 장소를 이름으로 직접 찾기 위해
US-03-03	비회원/회원	관광지 카드를 클릭해 상세 정보를 확인한다	방문 전 전화번호·주소·소개 등을 확인하기 위해

기능 명세

관광 DB 구축 [필수]

- 한국관광공사 국문 관광정보 서비스 API(TourAPI 4.0) 활용
- 수집 대상: 관광지(12), 문화시설(14), 레포츠(28), 숙박(32), 쇼핑(38), 음식점(39)
- 시도·시군구 코드 기준 전체 데이터 일괄 수집 후 DB 저장 (초기 1회 배치)
- API 직접 호출 최소화 — 수집 후 자체 DB에서 서비스

- 결측 데이터(이미지 없음, 주소 없음 등) 허용, 별도 처리 불필요

지역별·카테고리별 조회 [필수]

- 지역 필터: 시도(다중 선택 가능) → 시군구로 세분화
- 카테고리 필터: 콘텐츠 타입 코드 기준 (다중 선택 가능)
- 검색: 관광지명 키워드 검색 (부분 일치)
- 목록: 카드 2열 그리드, 페이지네이션(또는 무한 스크롤)
- 지도 연동: 목록의 장소가 네이버맵에 마커로 표시됨. 핀 클릭 시 InfoWindow, 상세 슬라이드 패널 연동

관광지 상세 [권장]

- 장소명, 카테고리, 주소, 전화번호, 대표 이미지, 소개(overview) 노출
- 지도에서 해당 마커 선택 시 목록에서 해당 카드 하이라이트

수용 기준

- AC-01-01: DB에 관광지 데이터가 적재되어 있으며 지역·카테고리 필터 결과가 정확히 반환됨
- AC-02-01: 시도 선택 후 해당 시도 소속 관광지만 노출됨
- AC-03-01: 카테고리 필터 선택 시 해당 콘텐츠 타입 코드에 매핑된 결과만 노출됨
- AC-03-02: 검색어 입력 시 장소명에 해당 키워드가 포함된 결과만 노출됨
- AC-03-03: 지역·카테고리 필터 미선택 시 전체 목록 반환됨

F05 — 일정 생성

사용자 스토리

ID	역할	행동	목적
US-05-01	회원	"+ 새 일정 만들기"를 클릭해 일정 생성 모달을 연다	새로운 여행 계획을 시작하기 위해
US-05-02	회원	여행 제목·출발일·귀환일·인원을 입력하고 일정을 만든다	여행의 기본 정보를 설정하기 위해
US-05-03	회원	내 일정 목록에서 기존 일정을 선택해 편집을 재개한다	이전에 만든 일정을 이어서 작업하기 위해

기능 명세

일정 생성 모달 [필수]

- 트리거: GNB "내 일정" 탭 최초 진입 시 또는 "+ 새 일정 만들기" 클릭
- 필수 입력: 여행 제목(최대 30자), 출발일, 귀환일
- 선택 입력: 여행 인원(기본값 1)
- 날짜 선택 시 "총 N박 N일" 자동 계산 표시
- 필수 항목 미입력 시 "만들기" 버튼 disabled
- 유효성: 출발일 ≤ 귀환일

일정 목록 [필수]

- 회원별 일정 목록을 최신 수정순으로 정렬 표시
- 활성 일정 전환: 일정명 드롭다운으로 다른 일정으로 전환 가능

수용 기준

- AC-05-01: 여행 제목 미입력 시 만들기 버튼 비활성화
- AC-05-02: 귀환일을 출발일보다 빠르게 선택 시 오류 표시
- AC-05-03: 일정 생성 완료 후 화면 B(내 일정)로 자동 이동되며 새 일정이 활성화됨

- AC-05-04: 기존 일정 선택 시 해당 일정의 후보군과 타임라인이 로드됨

F06 — 후보군 등록

사용자 스토리

ID	역할	행동	목적
US-06-01	회원	관광지 카드에서 "일정에 추가 +"를 클릭해 후보군에 등록한다	가고 싶은 장소를 일단 모아두기 위해
US-06-02	회원	등록된 후보군이 도시(시군구)별로 자동 묶여 표시되는 것을 확인한다	어느 도시에 어떤 장소를 방문할지 한눈에 파악하기 위해
US-06-03	회원	활성 일정이 없는 상태에서 후보군 추가를 시도하면 일정 선택/생성을 안내받는다	혼란 없이 올바른 플로우로 유도되기 위해

기능 명세

후보군 추가 [필수]

- 활성 일정이 있을 때: 클릭 즉시 후보군 등록 + 카드 상태 **Candidate** 로 전환
- 활성 일정이 없을 때: 일정 선택 드롭다운 또는 "새 일정 만들기" 유도
- 동일 장소 중복 추가 불가 (이미 추가된 경우 버튼 "추가됨" 비활성)

도시별 자동 분류 [필수]

- 후보군은 장소의 시군구 코드 기준으로 도시 그룹으로 묶임
- 도시 그룹은 CityGroup 컴포넌트로 표시 (도시명 + 장소 수 + 접기/펼치기)
- 장소 추가 순서대로 그룹 내 정렬

후보군 삭제 [권장]

- CandidateCard 우측 삭제 아이콘으로 후보군에서 제거
- 이미 타임라인에 배치된(Placed) 장소는 후보군 삭제 시 타임라인 블록도 함께 제거 여부 확인 모달 표시

수용 기준

- AC-06-01: 후보군 추가 후 AttractionCard가 **Candidate** 상태로 변경되고 "추가됨" 버튼이 비활성화됨
- AC-06-02: 내 일정 탭의 사이드바에 추가한 장소가 도시별로 그룹화되어 표시됨
- AC-06-03: 같은 장소를 두 번 추가 시도 시 중복 추가되지 않음

F07 — 일정 확정 (드래그 앤 드롭)

사용자 스토리

ID	역할	행동	목적
US-07-01	회원	CandidateCard를 타임라인 시간 슬롯으로 드래그 앤 드롭하여 일정에 배치한다	하루 일정을 직관적이고 시각적으로 구성하기 위해
US-07-02	회원	이미 배치된 PlaceBlock을 다른 날짜·시간 슬롯으로 드래그하여 이동한다	일정을 유연하게 조정하기 위해
US-07-03	회원	PlaceBlock의 X 버튼을 클릭하거나 후보군 사이드바로 드래그하여 일정에서 장소를 제거한다	일정에서 불필요한 장소를 빼기 위해
US-07-04	회원	날짜 탭을 클릭하여 여행 일자별 타임라인을 확인한다	여행 기간의 각 날짜 일정을 독립적으로 관리하기 위해
US-07-05	회원	같은 장소를 여러 날짜·시간대에 중복 배치한다	동일 장소를 여러 번 방문하는 일정을 구성하기 위해
US-07-06	회원	PlaceBlock 하단 핸들을 드래그하여 체류 시간을 조정한다	방문 시간을 현실에 맞게 세밀하게 설정하기 위해

기능 명세

날짜 탭 [필수]

- 일정 생성 시 날짜 범위 기준으로 Day 1 ~ Day N 탭 자동 생성
- 활성 탭: 보라 배경 + 흰색 텍스트

드래그 앤 드롭 [필수]

- 후보군 CandidateCard → 타임라인 DropZone으로 드래그
- 드래그 오버 시 DropZone 활성 상태 표시 (보라색 점선 테두리 + 배경색)
- 드롭 완료 시: PlaceBlock 생성 + CandidateCard Placed 상태(반투명)로 전환
- 중복 방문 허용: 이미 배치된 CandidateCard도 재드래그하여 같은 장소를 여러 번 배치 가능
- PlaceBlock 재배치: 다른 날짜 또는 다른 시간대 슬롯으로 드래그 앤 드롭으로 이동

PlaceBlock 삭제 [필수]

- PlaceBlock hover 시 우측 상단에 X 아이콘 노출, 클릭 시 삭제
- PlaceBlock을 좌측 후보군 사이드바로 드래그하면 사이드바가 빨간 삭제 존으로 변하고 드롭 시 삭제

체류 시간 조정 [필수]

- PlaceBlock 하단에 리사이즈 핸들 노출 (hover 시 표시)
- 핸들을 아래로 드래그 → 30분 단위로 스냅하며 체류 시간(duration_minutes) 조정
- 마우스 업 시 서버에 자동 저장

PlaceBlock 상세 편집 [권장]

- 클릭 시 메모 입력 가능
- 시작 시간 직접 수정 가능 (이후 블록 시간 연쇄 이동 여부 선택)

수용 기준

- AC-07-01: CandidateCard 드래그 시작 시 해당 카드의 ghost 이미지가 커서를 따라 이동함
- AC-07-02: 드롭 완료 후 타임라인에 PlaceBlock이 생성되고 CandidateCard가 반투명·취소선 상태로 변경됨

- AC-07-03: PlaceBlock X 클릭 후 블록 삭제 + CandidateCard가 정상 상태로 복원됨
- AC-07-04: Day 탭 전환 시 해당 날짜의 타임라인만 표시됨
- AC-07-05: 배치된(Placed) CandidateCard를 재드래그하면 동일 장소의 새 블록이 추가됨 (중복 방문)
- AC-07-06: PlaceBlock을 사이드바로 드래그 시 사이드바가 삭제 존으로 표시되고 드롭 시 블록 삭제됨
- AC-07-07: 체류 시간 핸들 드래그 시 30분 단위로 스냅되며 마우스 업 시 서버에 저장됨

F08 — 이동 시간 자동 계산 및 이동수단 선택

사용자 스토리

ID	역할	행동	목적
US-08-01	회원	타임라인에 장소를 배치하면 이전·이후 장소와의 이동 시간이 자동으로 표시된다	별도로 이동 시간을 검색하지 않아도 현실적인 일정을 짜기 위해
US-08-02	회원	이동 수단(대중교통 / 자동차 / 도보)과 소요 시간을 TransitPill에서 확인한다	이동 방법과 시간을 한눈에 파악하기 위해
US-08-03	회원	TransitPill을 클릭해 구간별 이동수단(대중교통·자동차·도보)을 변경하고 소요 시간·택시비를 즉시 확인한다	구간마다 다른 이동수단을 유연하게 선택하기 위해
US-08-04	회원	여행 일정 생성 시 주요 이동수단(대중교통·자동차·도보)을 설정하면 신규 블록 배치에 자동 적용된다	반복 변경 없이 자신에게 맞는 기본 수단으로 일정을 짜기 위해
US-08-05	회원	"지도로 보기" 버튼을 눌러 해당 날의 장소 위치와 이동 경로를 지도에서 확인한다	동선의 효율성을 시각적으로 파악하기 위해

기능 명세

이동 시간 자동 계산 및 삽입 [필수]

- 트리거: PlaceBlock 배치 / 재배치 / 삭제 시
- 여행의 기본 이동수단(defaultTransitMode) 기준으로 API 호출
- 대중교통: ODSay API / 자동차·도보: T Map API (SKT)
- 계산 결과를 PlaceBlock 사이에 TransitPill로 자동 삽입
- TransitPill 표시: 소요 시간(분) + 이동 수단

구간별 이동수단 변경 [필수]

- TransitPill 클릭 → 드롭다운 (대중교통 🚌 / 자동차 🚗 / 도보 🚶) 표시
- 드롭다운 오픈 시 3가지 모드 결과를 병렬 조회하여 소요 시간 표시
- 자동차 모드: 소요 시간 + T Map 택시 예상 요금(₩) 함께 표시
- 선택 클릭 → 해당 구간 블록 업데이트 + 자동 저장

여행 기본 이동수단 설정 [필수]

- 일정 생성 모달에서 기본 이동수단 선택 (대중교통 / 자동차 / 도보)
- 기본값: 대중교통
- 신규 블록 배치 시 해당 모드로 이동 시간 자동 계산

API 호출 최적화 [필수]

- 캐시 키: (출발지, 도착지, 출발 시간대, 이동수단 모드) 조합으로 모드별 독립 캐시
- 캐시 히트 시 API 호출 없이 즉시 반환
- 시간대별 캐시 정밀도는 관리자 설정(1~5단계)으로 조정 가능
- API 오류 또는 경로 없음: "경로 없음" 표시, 일정 저장은 정상 가능

경로 지도 시각화 [필수]

- 톨바 "지도로 보기" 버튼 → 우측 슬라이드인 지도 패널
- 네이버 지도에 날짜별 장소 마커(번호 순) 표시
- T Map route_coords 기반 실제 경로 폴리라인 표시 (자동차·도보 모드)
- 대중교통 모드: 구간별 노선 폴리라인(지하철·버스, lane별 색상) + 탑승/하차·환승 마커 표시
- 대중교통 환승·출발/도착 도보 구간은 인접 정류장 좌표 기준 T Map 보행 경로(초록 점선)로 표시
- 경로 데이터 없는 구간은 직선 점선으로 대체 표시

수용 기준

- AC-08-01: PlaceBlock 배치 후 1~2초 이내 TransitPill이 자동 삽입됨
- AC-08-02: 동일 구간 재배치 시 API 재호출 없이 캐시된 시간이 즉시 표시됨
- AC-08-03: API 응답 실패 시 "경로 없음" 표시 + 나머지 일정은 정상 동작
- AC-08-04: PlaceBlock 삭제 시 인접 TransitPill이 제거되고 새 인접 구간의 이동 시간이 자동 계산됨
- AC-08-05: TransitPill 클릭 → 드롭다운에서 이동수단 변경 → 즉시 저장 및 시간 업데이트
- AC-08-06: 자동차 모드 선택 시 소요 시간과 택시 예상 요금이 함께 표시됨
- AC-08-07: "지도로 보기" 클릭 시 해당 날짜의 장소 마커 및 이동 경로가 지도에 표시됨

F09 — 일정 저장·불러오기

사용자 스토리

ID	역할	행동	목적
US-09-01	회원	일정을 저장하여 나중에 다시 편집을 재개한다	한 번에 완성하지 않고 여러 세션에 걸쳐 계획을 세우기 위해
US-09-02	회원	저장된 일정 목록에서 원하는 일정을 선택한다	여러 여행 계획을 독립적으로 관리하기 위해

기능 명세

자동 저장 [필수]

- PlaceBlock 배치·이동·삭제 등 타임라인 변경 시 자동 저장 (debounce 2초)
- 저장 상태 표시: GNB 또는 저장 버튼 옆에 "저장됨" / "저장 중..." 표시

저장 범위 [필수]

- 저장 대상: 일정 기본 정보(제목·날짜·인원) + 후보군 목록 + 타임라인 블록(날짜·순서·시간·메모)
- 이동 시간 데이터는 별도 저장 (TransitCache 테이블 또는 블록 메타데이터)

수용 기준

- AC-09-02: 페이지 재접속 후 동일 일정 선택 시 이전 타임라인 상태가 복원됨
- AC-09-03: 로그인 세션이 만료되기 전에 작성 중이던 내용이 자동 저장됨

F10 — 여행 일정 공유게시판

사용자 스토리

ID	역할	행동	목적
US-10-01	회원	완성된 일정을 게시판에 공유한다	내 여행 경험을 다른 사람과 나누기 위해
US-10-02	비회원/회원	다른 사람이 공유한 일정을 열람한다	여행지 아이디어를 얻기 위해
US-10-03	회원	내가 작성한 게시글을 수정·삭제한다	잘못된 정보를 수정하거나 게시글을 내리기 위해

기능 명세

게시글 작성 [필수]

- 내 일정 중 하나를 선택하여 게시판에 공유
- 게시글 구성: 제목, 본문, 연결된 일정 (PlaceBlock 미리보기)
- 게시 시 해당 일정의 is_public 플래그 활성화
- 공유 중인 일정은 삭제 불가 (게시글을 먼저 삭제해야 일정 삭제 가능)

게시글 목록 [필수]

- 최신순 정렬 (기본)
- PostCard: 작성자 아바타·닉네임·날짜, 제목, MiniBlock 미리보기, 조회수·좋아요 수
- 비회원 포함 전체 공개

게시글 상세 [필수]

- 공유된 일정을 뷰어 형태로 표시 (편집 불가)
- 조회수 자동 증가

게시글 수정·삭제 [필수]

- 작성자 본인만 수정·삭제 가능
- 관리자는 모든 게시글 삭제 가능

좋아요 [권장]

- 회원만 좋아요 가능, 1회 제한

수용 기준

- AC-10-01: 비회원이 게시판 목록 및 게시글 상세 열람 가능
- AC-10-02: 비회원이 게시글 작성 시도 시 로그인 유도
- AC-10-03: 다른 사람의 게시글에 수정·삭제 버튼이 노출되지 않음
- AC-10-04: 게시글 상세 접근 시 조회수가 1 증가함
- AC-10-05: 게시글이 존재하는 일정 삭제 시도 시 "공유된 게시글이 있어 삭제할 수 없습니다" 오류 반환
- AC-10-06: 작성자 탈퇴 후에도 게시글이 보존되며 작성자명이 "탈퇴한 사용자"로 표시됨

F13 — 관광지 AI 챗봇

관광지 상세에서 자연어로 질문하고 주변 장소 추천까지 받는 대화형 도우미. Spring AI + gms(OpenAI 호환 프록시) 기반.

사용자 스토리

ID	역할	행동	목적
US-13-01	회원	관광지 상세에서 해당 장소에 대해 자연어로 질문한다	운영시간·특징 등 궁금증을 빠르게 해소하기 위해
US-13-02	회원	이어지는 질문으로 대화를 계속한다 (멀티턴)	맥락을 유지한 채 추가로 물어보기 위해
US-13-03	회원	"주변에 가볼 만한 곳"을 물어 추천을 받는다	동선 내 다른 장소를 함께 계획하기 위해
US-13-04	회원	답변에 언급된 주변 장소 버튼을 눌러 지도 핀·상세로 이동한다	추천 장소를 바로 확인·탐색하기 위해

기능 명세

관광지 컨텍스트 Q&A [권장]

- POST /api/attractions/{id}/chat — 해당 관광지 정보(이름·분류·주소·운영정보·소개 등)를 시스템 프롬프트로 주입
- 모델: gms(OpenAI 호환 프록시)의 gpt-4.1 , 키는 환경변수 GMS_KEY 로 주입
- 멀티턴: conversationId 기준으로 대화 맥락 유지 (Spring AI ChatMemory)
- AI 토큰 비용으로 인해 로그인(회원) 필요

주변 장소 추천 [권장]

- 관광지 좌표 기준 반경 약 3km·최대 8곳을 DB에서 거리순 조회(ST_Distance_Sphere)해 컨텍스트로 주입
- 응답에 실제 언급된 장소만 버튼으로 노출 → 클릭 시 지도 핀 + "상세정보 보기" 인포윈도우
- '상세정보 보기'로 진입 시 네비게이션 스택에 push, 뒤로가기로 이전 장소·대화·스크롤 위치까지 복원

수용 기준

- AC-13-01: 비로그인 사용자가 챗 호출 시 "로그인 후 이용" 안내 노출
- AC-13-02: 같은 대화에서 이전 질문의 맥락이 유지됨
- AC-13-03: 주변 질문 시 반경 내 실제 DB 장소(음식점·관광지 등)가 거리순으로 추천됨
- AC-13-04: 답변에 언급된 장소만 버튼으로 노출되며, 클릭 시 지도 핀 + 상세 이동
- AC-13-05: 주변 상세로 이동 후 뒤로가기 시 이전 장소의 대화·아코디언·스크롤이 복원됨

5. 기능 요구사항 — Phase 2 (고도화)

MVP 안정화 이후 구현. 현재 Sprint에서는 설계·구현 범위에 포함하지 않는다.

G01 — 실시간 공동 편집

ID	역할	행동	목적
US-G01-01	회원	공유 링크로 일행을 일정에 초대한다	같이 여행하는 사람과 일정을 함께 편집하기 위해
US-G01-02	회원	일정 화면에서 일행의 편집 내용이 실시간으로 반영되는 것을 확인한다	서로 논의하며 일정을 완성하기 위해

기술 요건: Spring WebSocket + STOMP, 동시 편집 충돌 처리(Optimistic Locking 또는 OT 알고리즘 검토 필요)

6. 비기능 요구사항

6-1. 성능

항목	목표
관광지 목록 API 응답 시간	500ms 이하 (캐시 기준)
이동 시간 계산 API 응답 시간	2초 이하 (ODsay 의존)
드래그 앤 드롭 인터랙션	60fps 유지 (jank 없음)
초기 페이지 로드	3초 이하

6-2. 보안

항목	요구사항
인증	JWT (Access 30분 + Refresh 7일), HttpOnly 쿠키
비밀번호	BCrypt 해시 저장, 평문 저장 절대 금지
SQL Injection	MyBatis #{ } 바인딩 필수
인가	모든 보호 API에 서버 측 소유권 검증
API 키	환경 변수 관리, 소스코드에 하드코딩 금지
CORS	허용 Origin 명시적 설정

6-3. 사용성

항목	요구사항
반응형	데스크톱(1280px 이상) 우선. 태블릿(768px) 기본 지원
로딩 상태	API 호출 중 스피너 또는 스켈레톤 UI 표시
오류 상태	사용자 친화적 오류 메시지 (기술적 스택 트레이스 노출 금지)
빈 상태	데이터 없음 상태(Empty State) 화면 구성

6-4. 유지보수성

항목	요구사항
코드 컨벤션	docs/03_dev/conventions.md 준수
테스트	핵심 비즈니스 로직(이동 시간 계산, 즐겨찾기 자동 연동) 단위 테스트 작성

7. 제약 사항 및 가정

구분	내용
외부 API 의존	ODsay API 무료 플랜: 일 1,000회 쿼리 제한. 캐싱 전략 필수. 초과 시 "이동 시간 계산 불가" graceful 처리
AI 챗봇 API	gms(OpenAI 호환 프록시)의 gpt-4.1 사용. 토큰 비용으로 챗 호출은 회원 전용. 대화 맥락은 인메모리(ChatMemory) 보관 — 서버 재시작 시 소멸
관광공사 API	초기 데이터 일괄 수집(배치) 후 자체 DB 운용. 실시간 API 호출 최소화
지도	네이버맵 API 연동. 마커 표시, 지도 이동(morph), InfoWindow, 후보군 마커 포함
숙박 예약 연동	Phase 2 이후로 분리. MVP에서는 숙박 카테고리 관광지 조회(관광공사 DB)만 제공
모바일 앱	MVP는 웹 브라우저 전용. 모바일 앱 미지원
국내 여행만 지원	해외 관광지 데이터 및 국제 이동 수단 계산은 제외
동시 접속자	개발 환경 기준 단일 서버. 실사용 규모 트래픽 대응은 배포 단계 이후 검토

8. 기능 우선순위 최종 확인

Phase 1 — MVP 구현 순서 (권장)

순서	기능 ID	기능명	이유
1	F12	회원 관리	모든 개인화 기능의 전제 조건
2	F01·F02·F03	관광지 DB 구축 및 조회	서비스의 콘텐츠 기반, 이후 기능의 데이터 소스
4	F05	일정 생성	후보군·타임라인 기능의 컨테이너
5	F06	후보군 등록	드래그 앤 드롭 이전 단계
6	F07	일정 확정 (드래그 앤 드롭)	핵심 UX. 난이도 높음, 충분한 개발 시간 필요
7	F08	이동 시간 자동 계산	F07과 병행 구현
8	F09	일정 저장·불러오기	F07·F08 완성 후 통합
9	F10	공유게시판	커뮤니티 도메인
11	F13	관광지 AI 챗봇	부가 기능. 관광지 조회·지도 기반 위에 확장 (권장)

일정 압박 시 범위 조정 기준

기능	축소 방향
F07 드래그 앤 드롭	드래그 앤 드롭 대신 목록에서 "날짜·시간 선택" 방식으로 단순화
F08 이동 시간 자동 계산	ODsay API 연동 실패 시 직선 거리 기반 예상 시간으로 대체
F10 좋아요	조회수만 제공, 좋아요 기능 미포함

Phase 2 우선순위

순위	기능 ID	기능명	조건
1	G01	실시간 공동 편집	Phase 1 전 기능 안정화 + WebSocket 설계 검토 완료 후

9. 구현 확장 요구사항 (MVP 이후)

Phase 1 MVP 안정화 후 실제 구현·반영된 추가 기능. 기획서 Phase 2(G01 실시간 공동 편집)를 포함해 고도화했다.

F14 — 실시간 협업 & 공유 링크 (Phase 2 G01 구현 완료)

ID	역할	행동	목적
US-14-01	회원	일정에 다른 회원을 협업자(EDITOR/VIEWER)로 초대한다	일행과 일정을 함께 편집하기 위해
US-14-02	회원	공유 링크(PRIVATE/VIEW/EDIT)를 발급해 전달한다	비회원·미초대자에게도 일정을 보여주기 위해
US-14-03	회원	다른 협업자의 커서·편집 내용을 실시간으로 본다	동시에 논의하며 일정을 완성하기 위해

기능 명세

- WebSocket(STOMP/SockJS) /ws + /topic/trip/{id} · /topic/trip/{id}/presence 구독으로 실시간 동기화
- 동시 편집 충돌은 trip_block.version **낙관적 락**으로 처리 (사용자 편집 시 +1, transit 재계산은 미변경)
- 협업자 역할: EDITOR(조회+편집), VIEWER(조회). 소유자(trip.member_id)와 별도 관리
- 공유 링크: share_access (PRIVATE/VIEW/EDIT) + URL-safe share_token . 비로그인도 ?s=token 으로 조회 가능
- 참가자 커서 위치·블록 잡기(grab lock)·색상 배정 표시

수용 기준

- AC-14-01: EDIT 공유 링크 수신자는 로그인 시 편집, VIEW는 읽기 전용으로 진입
- AC-14-02: 한 사용자의 블록 이동이 다른 참가자 화면에 실시간 반영됨
- AC-14-03: 두 사용자가 같은 블록을 동시 편집 시 version 불일치로 후행 저장기 거부되고 최신 상태로 재동기화됨

F15 — 마이페이지 & 개인 장소

기능	명세
마이페이지 셀	/mypage 하위 7개 탭(여행·내정보·내장소·방문지도·내가쓴글·북마크·좋아요)
프로필	프로필 사진 업로드/삭제, 닉네임 인라인 수정, 계정 유형(이메일/카카오) 표시
내 장소(member_place)	Kakao Local 검색으로 커스텀 장소 저장·재사용 → 후보군에 추가 가능 (좌표 기반 이동시간 계산)

F16 — 댓글·북마크 (커뮤니티 고도화)

기능	명세
댓글·대댓글	post_comment.parent_id 로 1단계 대댓글 지원. 작성자 탈퇴 시 SET NULL(보존), 부모 삭제 시 CASCADE
북마크	게시글 북마크. 글 소프트 딜리트 시에도 북마크 레코드 보존 → "삭제된 글입니다" 표시
소프트 딜리트	post.deleted_at 로 논리 삭제

F17 — 방문 지도 (여행 이야기 연동)

기능	명세
방문 지도	마이페이지에서 대한민국 SVG 지도에 방문(여행이야기 작성) 지역 하이라이트 + 진행률
지역 표지 사진	한 지역에 후기 여러 개일 때 member_map_cover 로 대표 표지 글 고정. 미선택 시 최신 후기 자동 표시

외부 연동 확장

연동	용도
T Map (SKT)	자동차 경로·택시 예상 요금, 도보 경로(GeoJSON LineString) — 기획서 ODsay 단독에서 확장
Kakao OAuth / Local	소셜 로그인 + 장소 키워드 검색
이미지 첨부(attach)	프로필·게시글·임시저장 이미지 메타데이터 통합 관리, 고아 이미지 정리 스케줄러

Use-Case 다이어그램

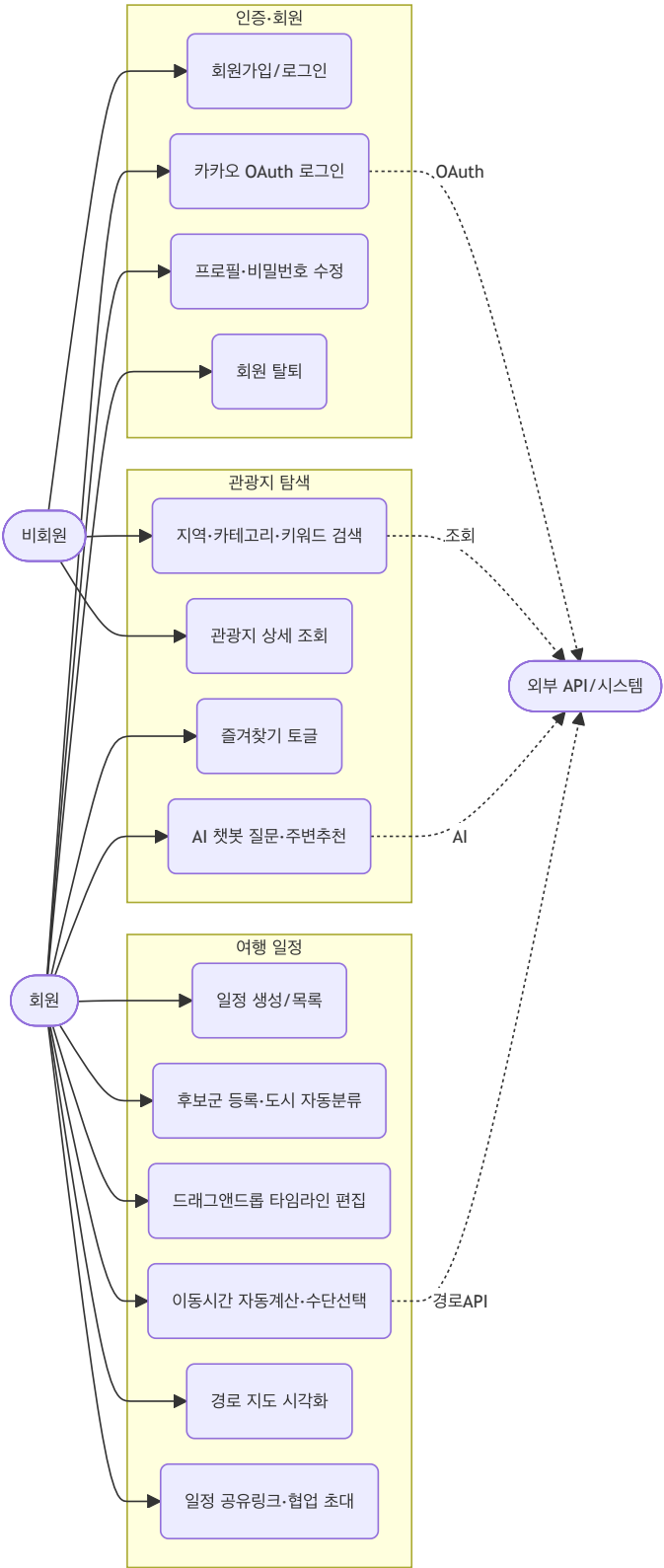
액터별 유스케이스 정의와 유스케이스 다이어그램.

1. 액터 정의

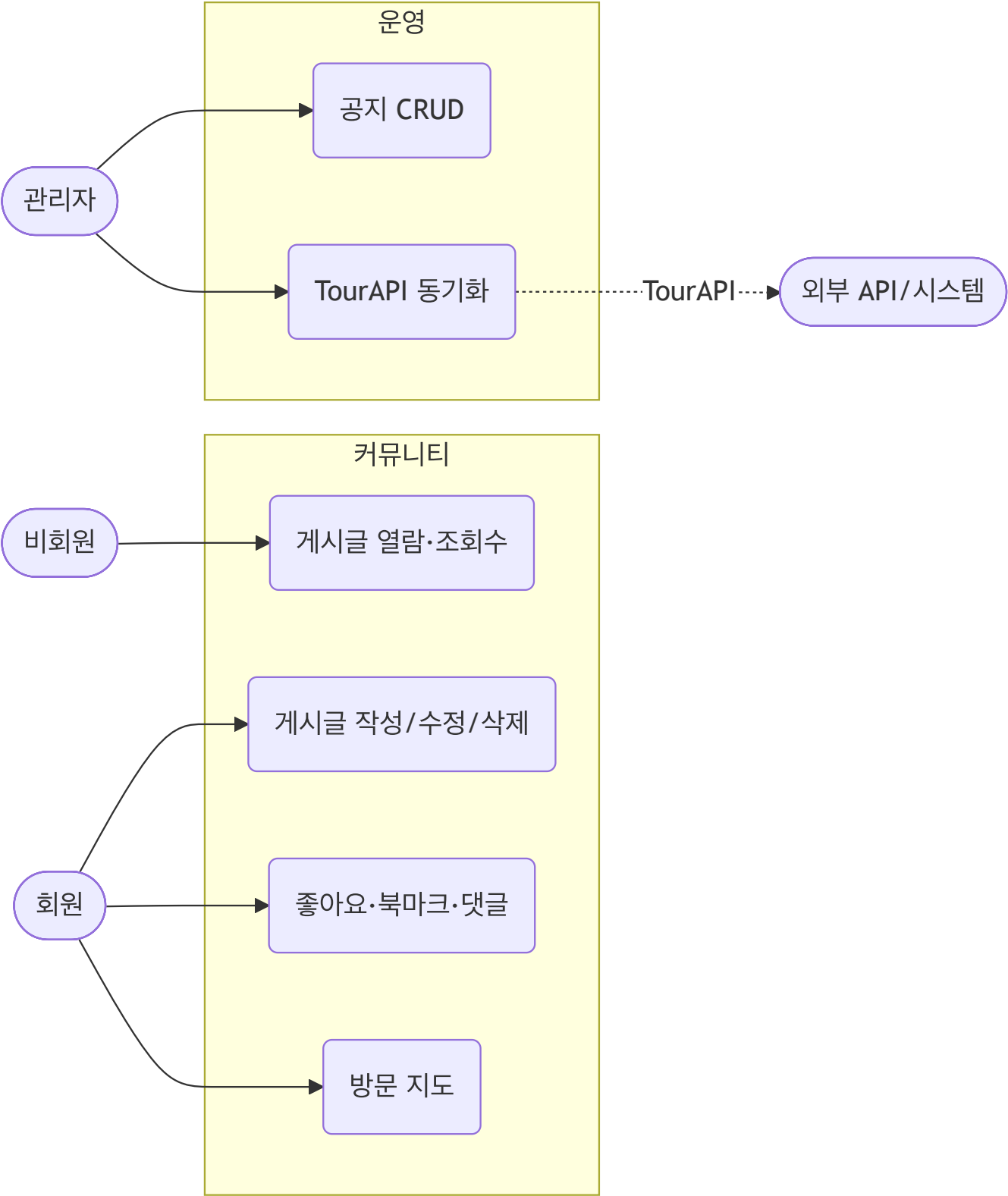
액터	설명	권한
비회원 (Guest)	미로그인 방문자	관광지 조회, 커뮤니티/공지 열람, 공유 링크(VIEW) 조회
회원 (Member)	로그인 사용자	Guest 권한 + 일정 CRUD·드래그 편집, 즐겨찾기, 후보군, 게시글/댓글/좋아요/북마크, AI 챗봇, 협업
관리자 (Admin)	운영자	Member 권한 + 공지 CRUD, 게시글 관리, TourAPI 동기화
시스템/외부 API	배치·연동 주체	TourAPI 수집, ODSay/T Map 경로 계산, Kakao OAuth/검색, gms(AI) 호출

2. 유스케이스 다이어그램

2-1. 회원 · 관광지 탐색 · 여행 일정



2-2. 커뮤니티 · 운영



3. 주요 유스케이스 명세 (요약)

ID	유스케이스	액터	사전조건	기본 흐름
UC10	후보군 등록	회원	활성 일정 존재	관광지 카드 "추가" → 후보군 등록 → 도시(시군구) 자동 그룹화 → 첫 등록 시 동일 도시 즐겨찾기 일괄 추가
UC11	타임라인 편집	회원	후보군 ≥ 1	CandidateCard를 Day 타임라인으로 드래그 → PlaceBlock 생성 → 체류시간 핸들 조정 → 자동 저장
UC12	이동시간 계산	회원, 외부	블록 ≥ 2	블록 배치/이동/삭제 트리거 → 캐시 조회 → 미스 시 ODsay(대중교통)/T Map(자동차·도보) 호출 → TransitPill 삽입
UC14	협업·공유	회원	일정 소유	공유 링크 발급(VIEW/EDIT) 또는 회원 검색 후 협업자(EDITOR/VIEWER) 초대 → WebSocket 실시간 동기화
UC8	AI 챗봇	회원, 외부	로그인	관광지 상세에서 질문 → 장소정보+반경3km 주변장소 컨텍스트 주입 → gms gpt-4.1 멀티턴 응답 → 언급 장소 핀 이동

전체 기능 요구사항(US-xx-xx, AC)은 요구사항 정의를 참조.

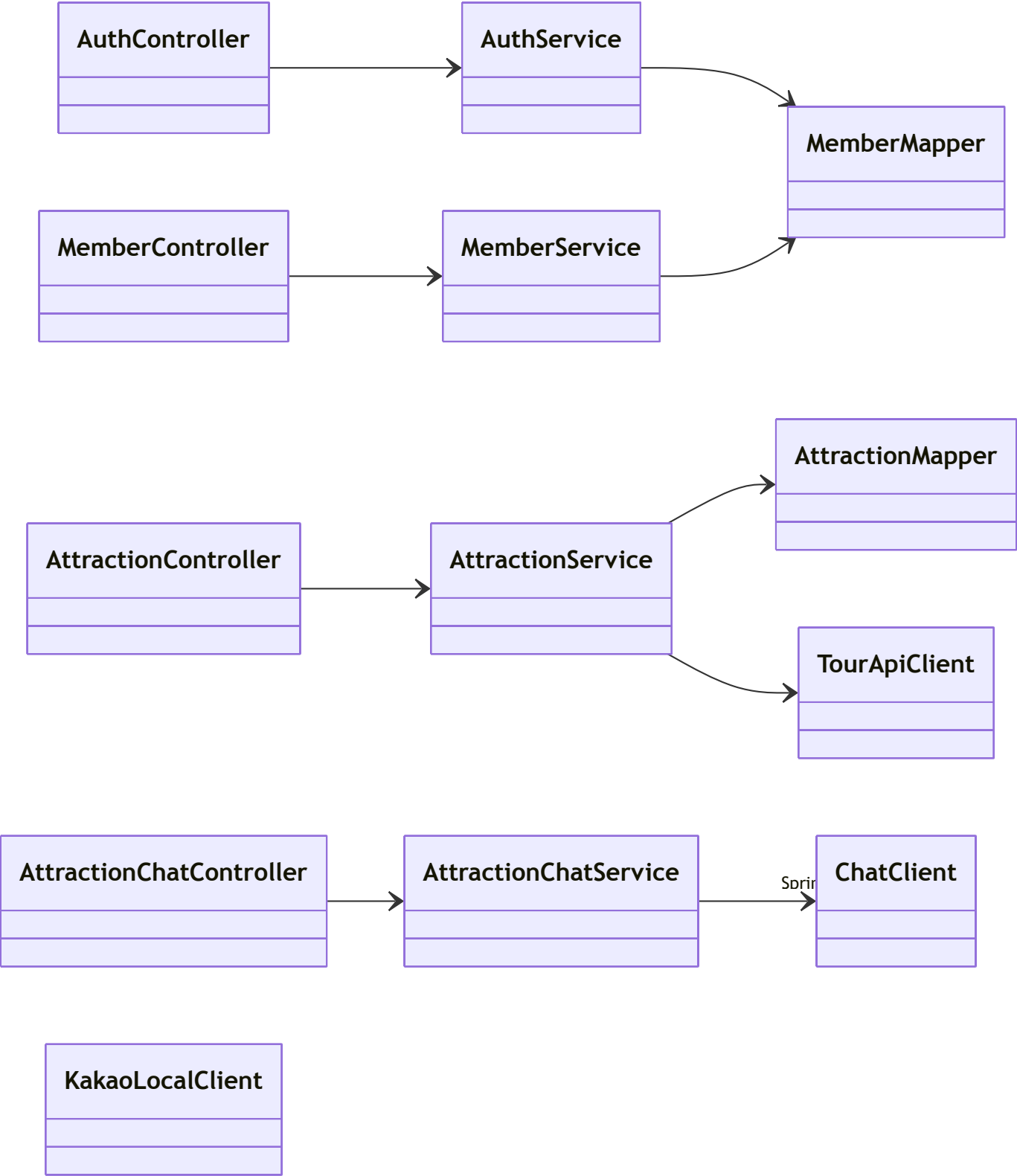
클래스 다이어그램 (백엔드)

도메인 중심 패키지 구조(member · attraction · plan · community · place · chat · global).
계층: **Controller** → **Service(Impl)** → **Mapper(MyBatis)** → **DB**, 외부 연동은 **Client** 클래스로 분리.

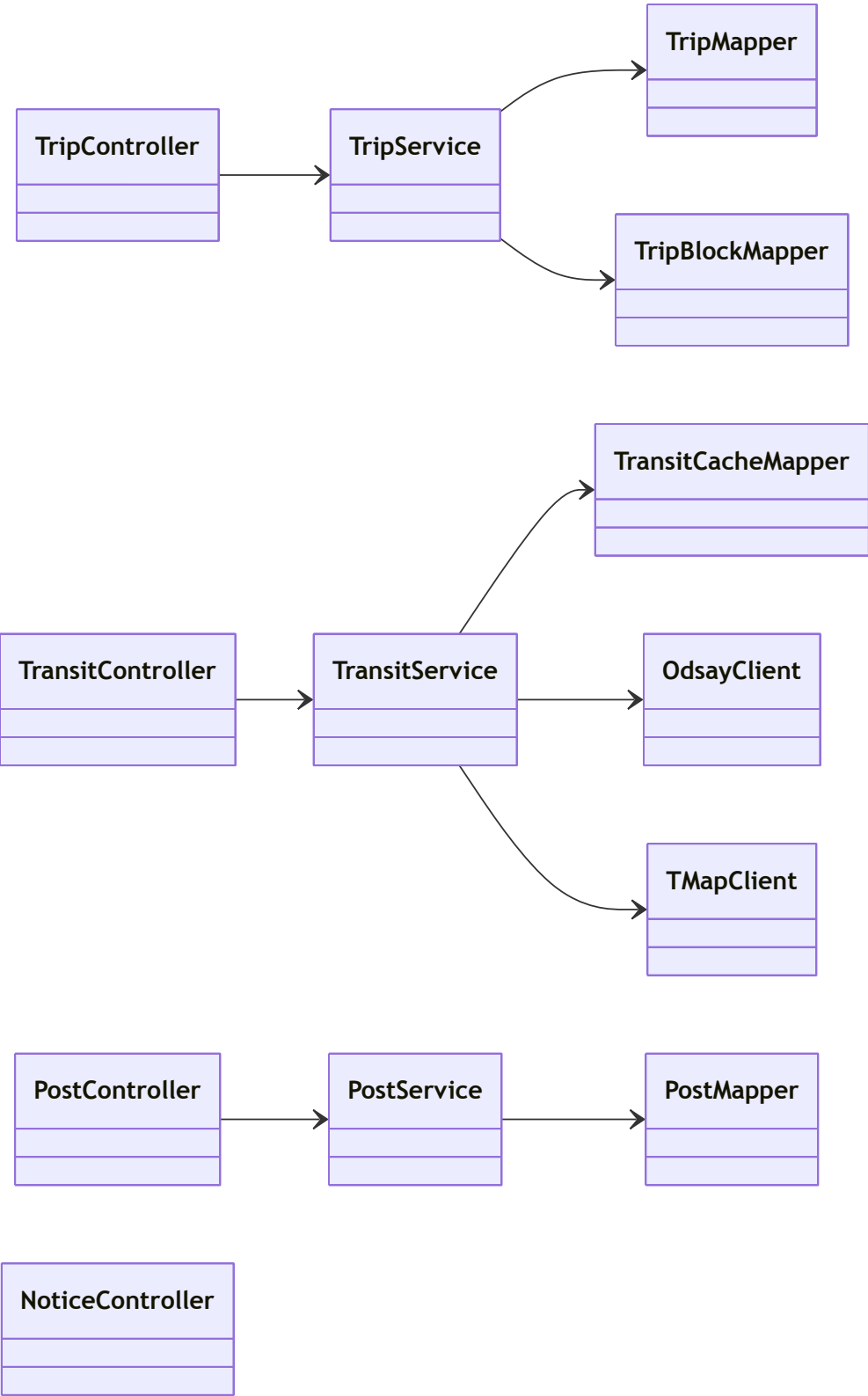
1. 계층형 아키텍처 (대표 도메인)

Controller → Service → Mapper/Client 계층을 도메인 그룹별로 나눠 표기한다.

1-1. 회원 · 관광지 · 챗봇 · 장소

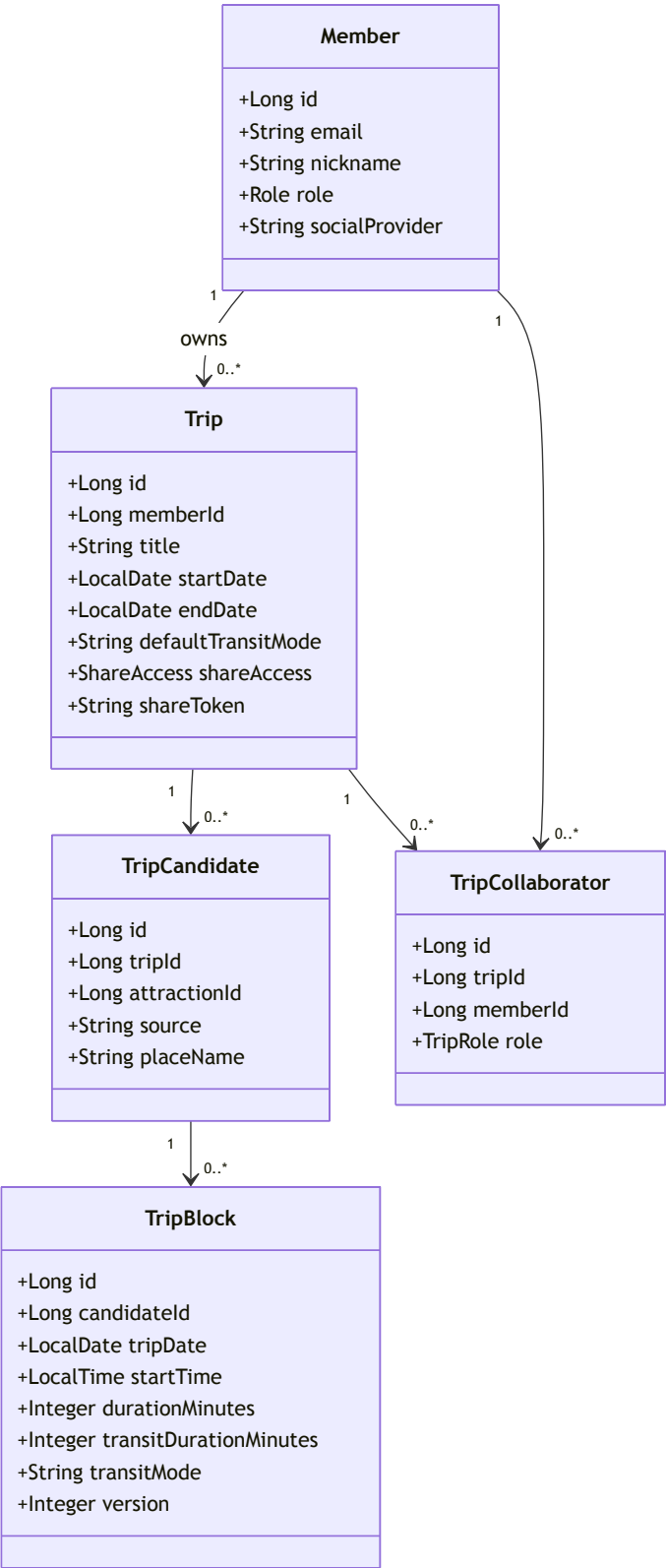


1-2. 여행 일정 · 이동 · 커뮤니티

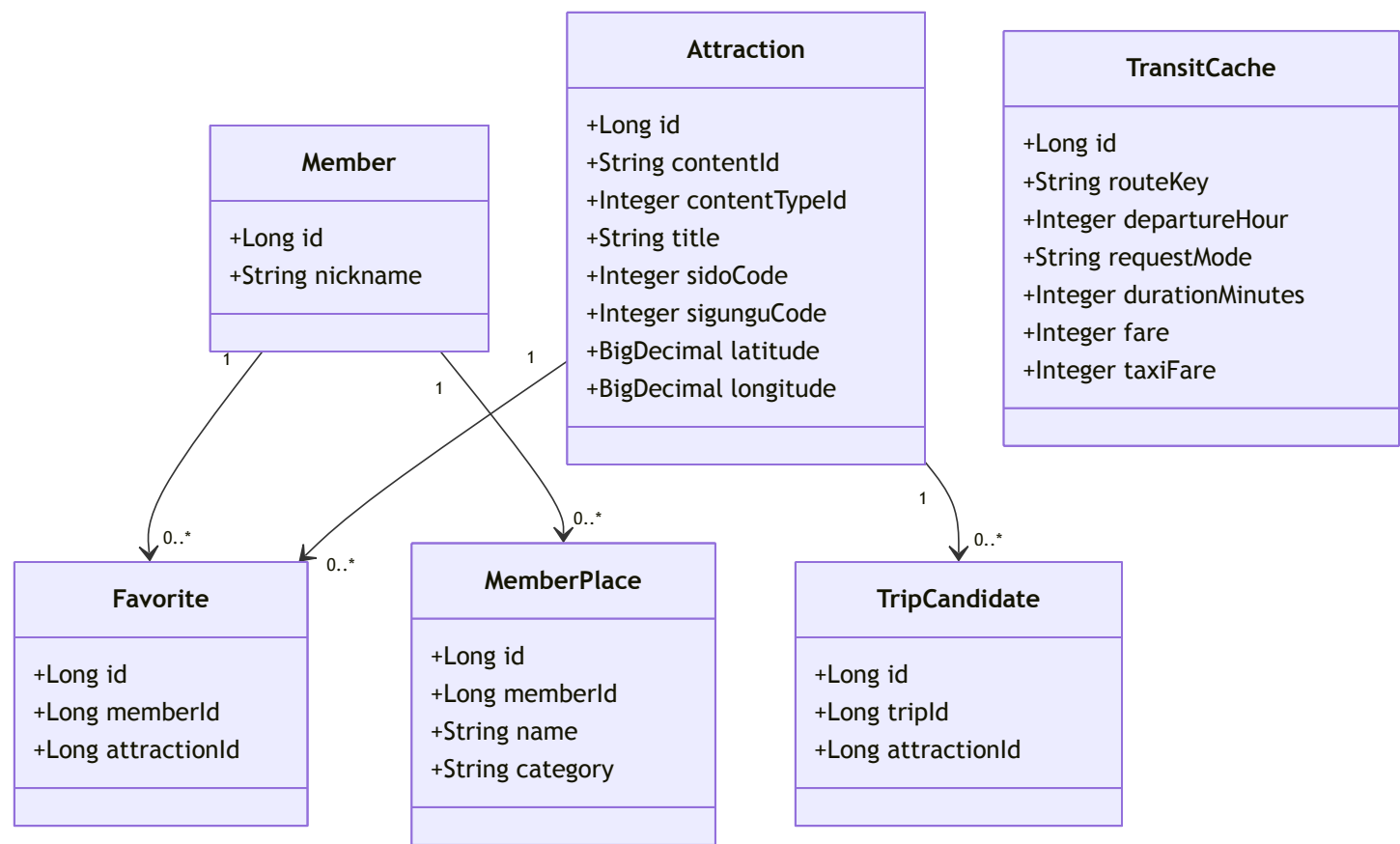


2. 도메인 모델 (엔티티 관계)

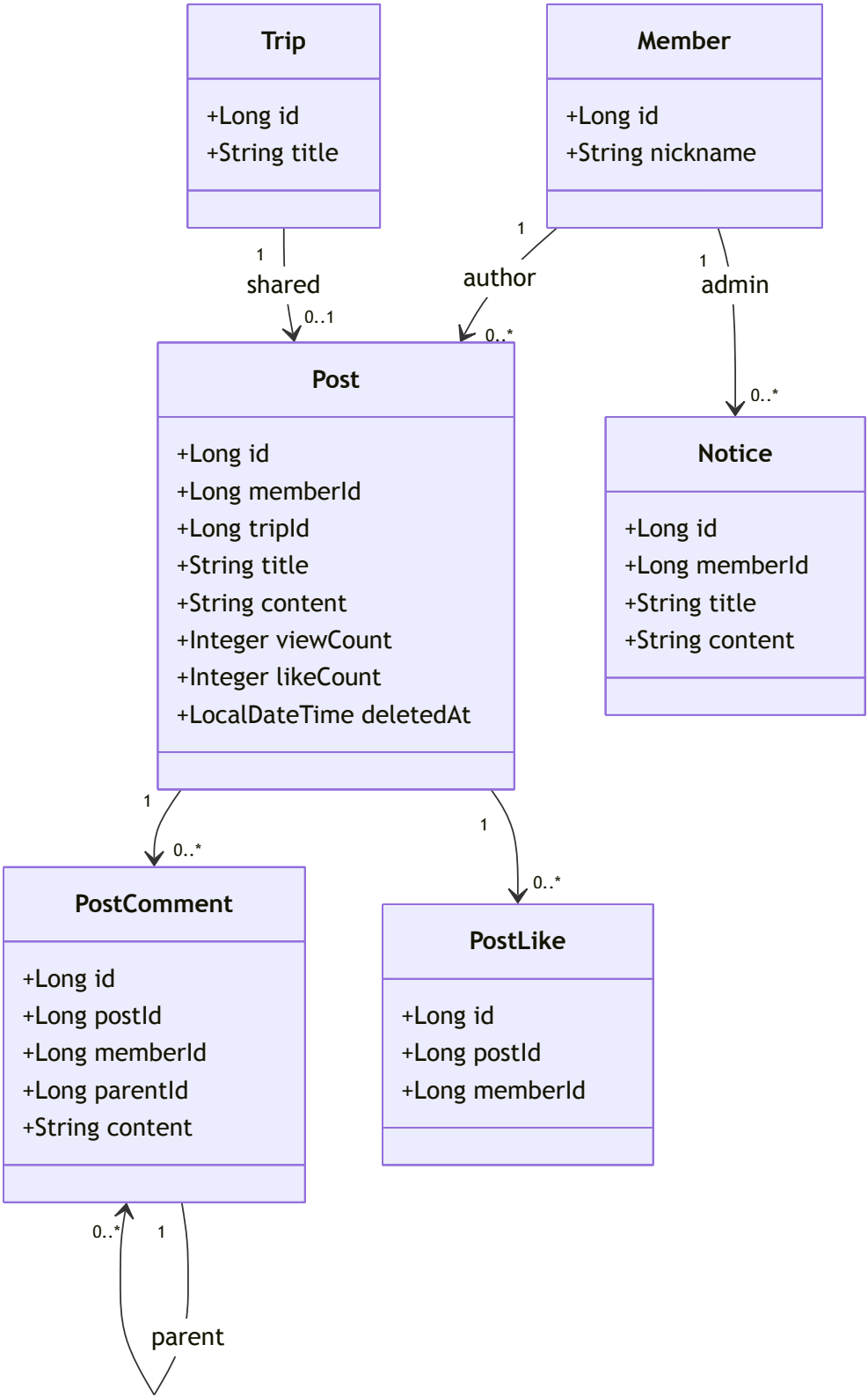
2-1. 회원 · 여행 일정



2-2. 관광지 · 이동 · 개인 장소



2-3. 커뮤니티



3. 패키지·책임 요약

패키지	Controller	Service	주요 Client/Config
member	AuthController, MemberController	AuthService, MemberService, MemberMapService	(Kakao OAuth)
attraction	AttractionController, AttractionSyncController	AttractionService	TourApiClient, TourApiSyncJobConfig, TourApiCallLimiter
chat	AttractionChatController	AttractionChatService	ChatClientConfig (Spring AI / gms)
plan	TripController, TransitController, TripPresenceController	TripService, TransitService	OdsayClient, TMapClient
community	PostController, CommentController, LikeController, BookmarkController, NoticeController	PostService, CommentService, LikeService, BookmarkService	PostImageCleanupListener
place	MyPlaceController, PlaceController	MemberPlaceService, PlaceService	KakaoLocalClient
global	ImageController	FileStorageService, OrphanImageCleanupScheduler	SecurityConfig, WebSocketConfig, JwtTokenProvider, GlobalExceptionHandler, ApiResponse

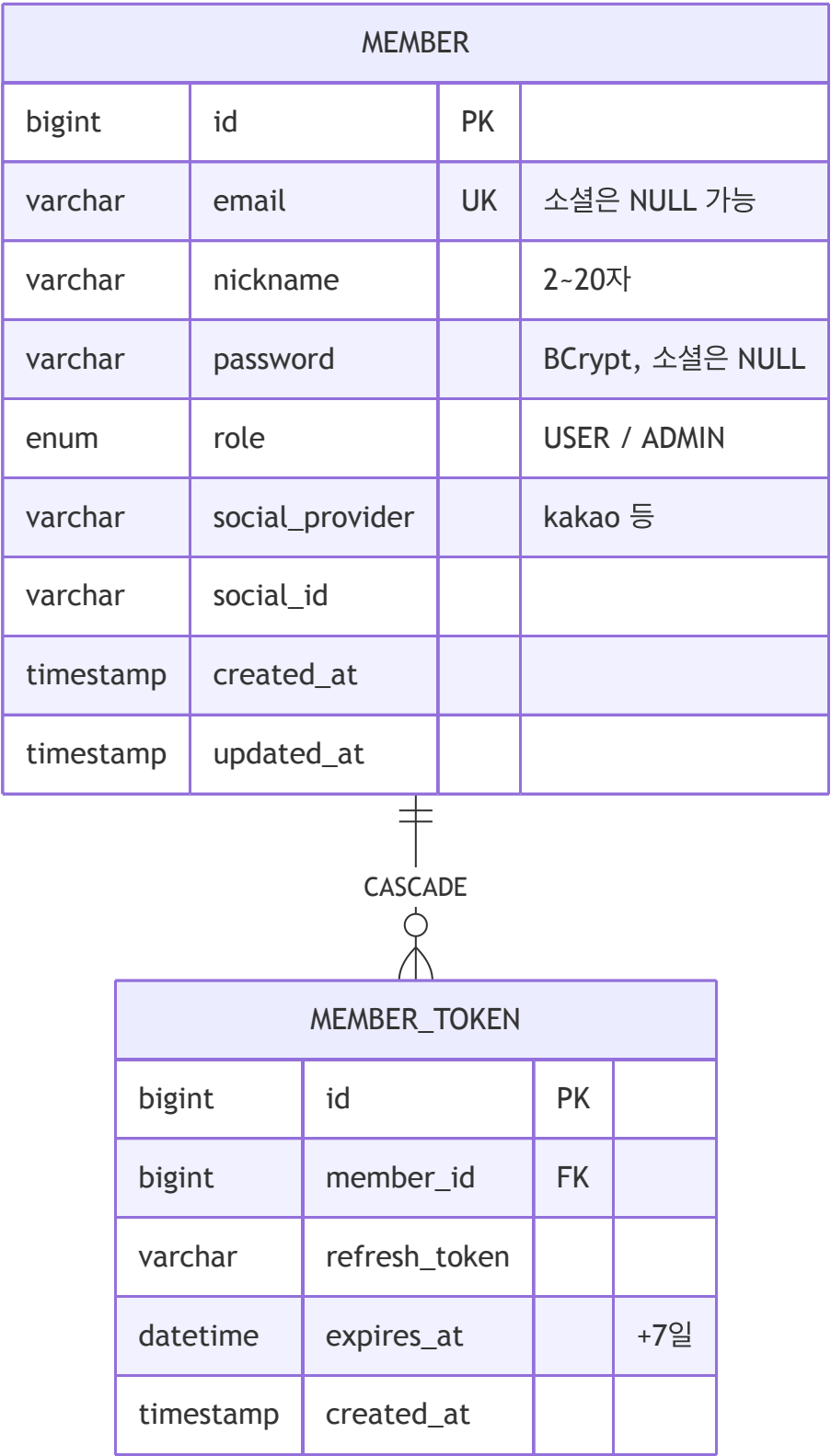
공통 응답은 ApiResponse<T> ({success, data, message, errorCode})로 래핑. 인증은 JwtAuthenticationFilter + JwtTokenProvider , 실시간 협업은 WebSocketConfig + STOMP(TripPresenceController) 기반.

ER 다이어그램

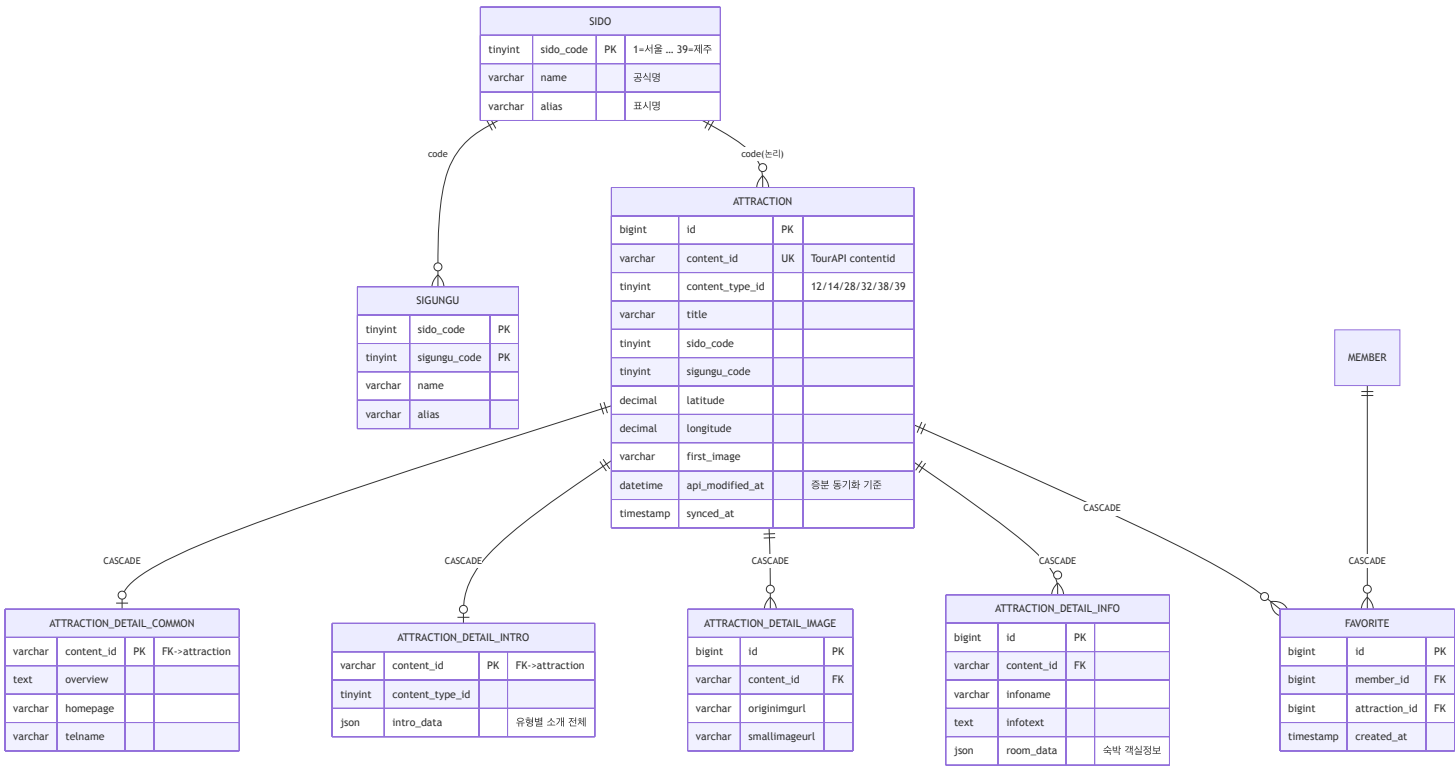
도메인 테이블 + 참조/설정 테이블. 모든 FK의 삭제 정책(CASCADE / SET NULL / RESTRICT)을 관계 라벨에 표기.
한 장에 모든 테이블을 그리면 가독성이 떨어져, **도메인별 4개 ERD**로 나눠 표기한다. (타 도메인에서 참조만 되는 테이블은 이름만 표시)

1. 도메인별 ERD

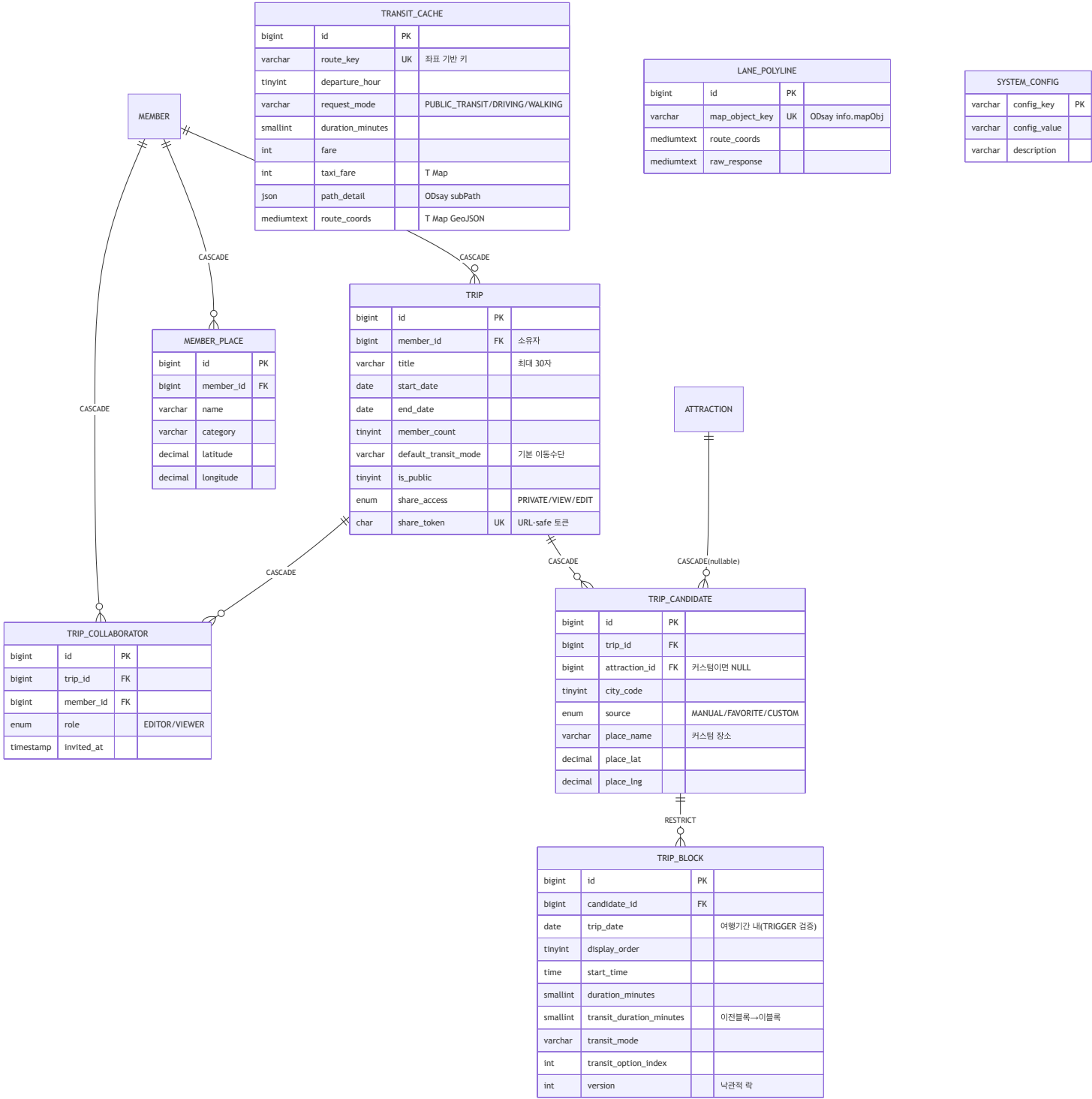
1-1. 회원 · 인증



1-2. 관광지 · 지역

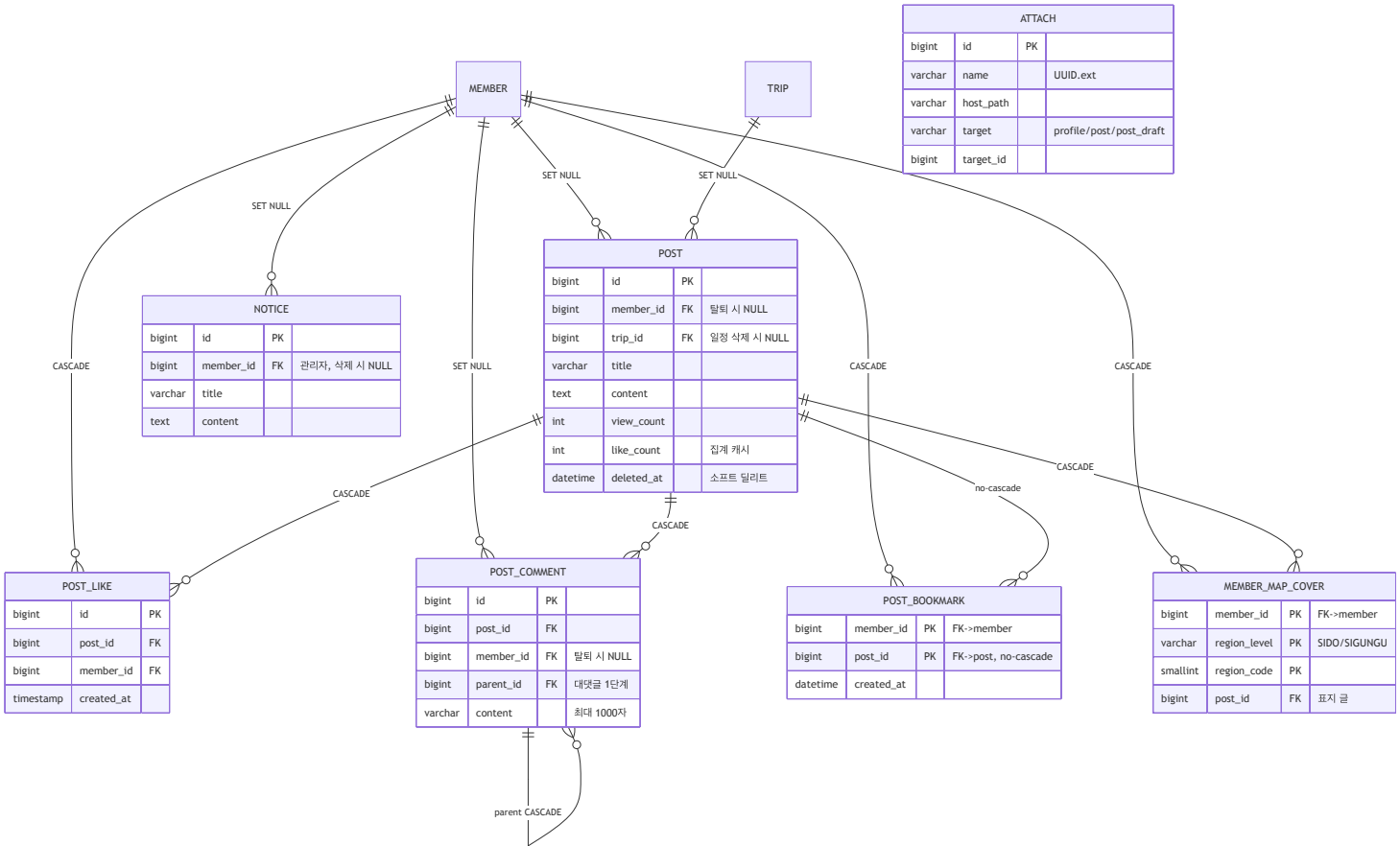


1-3. 여행 일정 · 이동



TRANSIT_CACHE · LANE_POLYLINE · SYSTEM_CONFIG 는 FK 없이 좌표 키·설정 키로 독립 운용되는 캐시/설정 테이블이다.

1-4. 커뮤니티 · 콘텐츠



Mermaid 표기상 핵심 컬럼만 발췌했다. 전체 DDL·인덱스·트리거·시드 데이터는 스키마 정본(`schema.sql`)을 따른다.

2. 핵심 설계 결정 (FK 삭제 정책)

테이블	정책	의도
member	하드 딜리트	탈퇴 시 앱 레이어가 trip_block → trip_candidate → trip 순으로 정리 후 회원 삭제
member_token , favorite , trip , member_place , trip_collaborator	ON DELETE CASCADE	회원 종속 데이터는 함께 제거
post , notice (member_id)	ON DELETE SET NULL	작성자 탈퇴해도 글·공지 보존 → "탈퇴한 사용자" 표시
post (trip_id)	ON DELETE SET NULL	공유 일정 삭제돼도 게시글 본문 보존
trip_block → trip_candidate	ON DELETE RESTRICT	후보군 삭제 전 "타임라인 블록도 삭제" 모달 확인 UX 보장
post_comment (parent_id)	ON DELETE CASCADE	부모 댓글 삭제 시 대댓글 동반 삭제 (1단계)
post_bookmark (post_id)	CASCADE 없음	글이 소프트 딜리트돼도 북마크 레코드 보존 → "삭제된 글입니다" 표시
attraction_detail_*	ON DELETE CASCADE (content_id)	관광지 재동기화/삭제 시 상세 동반 정리

3. 비정규화·캐시 전략

- **trip_block.transit_duration_minutes / transit_mode** : 이동 시간을 블록에 비정규화 저장 — 매 조회 시 외부 API 재호출 방지.
- **post.like_count** : post_like 집계를 캐시 컬럼으로 유지 — 목록 조회 시 COUNT 회피.
- **transit_cache** : (좌표 route_key, departure_hour, request_mode) 키로 ODSay·T Map 응답 캐시. 모드별 독립 캐시(UNIQUE).
- **lane_polyline** : ODSay 노선 형상은 거의 안 바뀌므로 map_object_key 단위 영구 캐시.
- **attraction** : TourAPI api_modified_at 기준 증분 배치 동기화. 상세(detail_*)는 지연 적재.

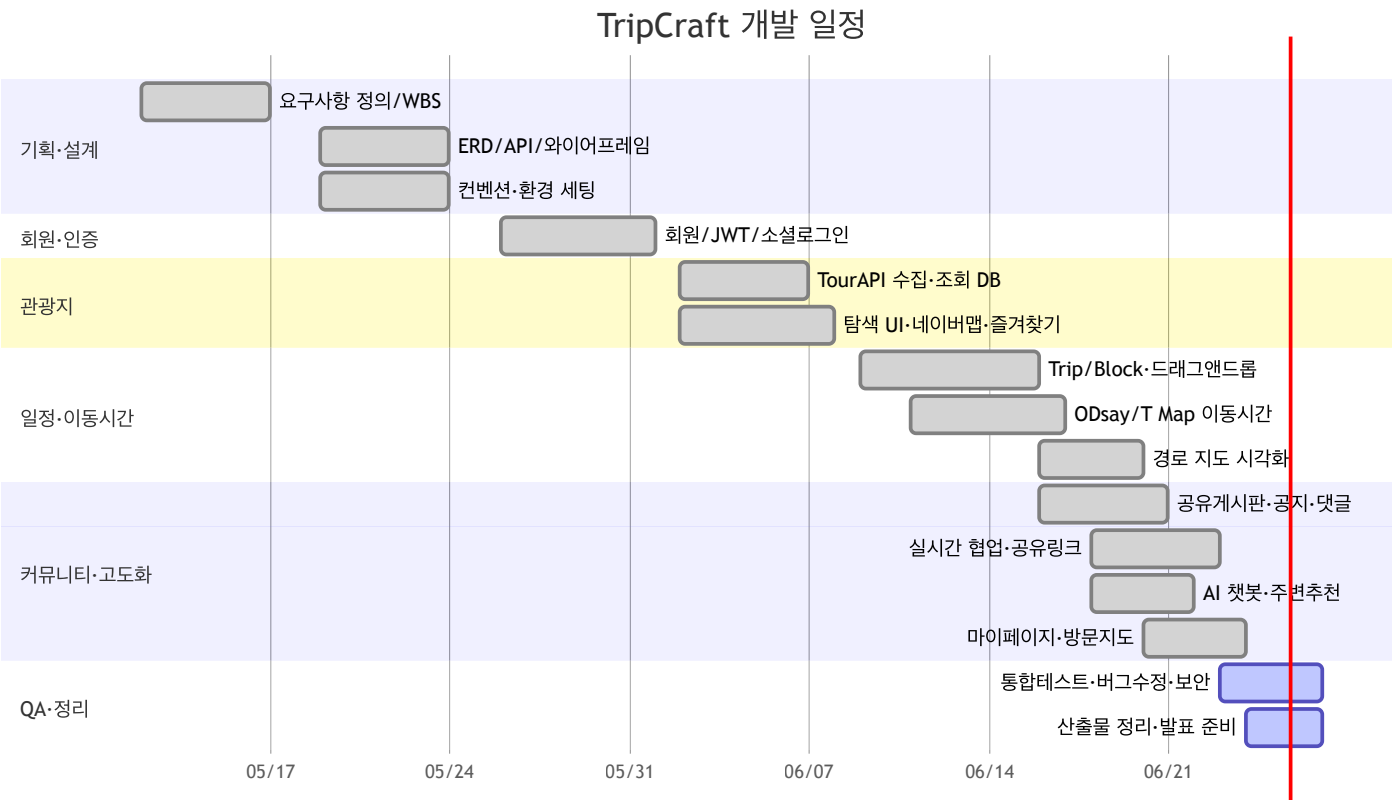
WBS & 간트 차트

기준 일정: 약 6주 · 개발 인원: 2인 (전진 · 송정기)

1. 전체 일정 요약

단계	기간	주요 내용
1단계 — 기획	Week 1	요구사항 정의, WBS, 협업 환경 세팅
2단계 — 설계	Week 2	ERD, API 명세, UI 와이어프레임, 코딩 컨벤션
3단계 — 개발	Week 3 ~ 5	회원 → 관광지·즐거찾기 → 일정·이동시간 구현
4단계 — 커뮤니티·고도화	Week 6 ~	공유게시판·공지, 협업·공유, AI 챗봇, 마이페이지
5단계 — QA·정리	막바지	통합 테스트, 버그 수정, 보안 점검, 산출물 정리

2. 간트 차트 (Mermaid)



3. 단계별 상세 (WBS)

1단계 — 기획 (Week 1)

- 레포지토리·브랜치 전략, 디렉터리 구조(backend/ · frontend/ · docs/), 역할 분담
- 사용자 스토리 작성, 요구사항 명세서(requirements.md), 기능 우선순위(Phase 1/2)
- WBS·간트 작성, 주차별 마일스톤 확정

2단계 — 설계 (Week 2)

- 시스템 구조도, Use-case 다이어그램
- ERD 설계·확정(회원·관광지·즐거찾기·일정·게시글·공지)
- 회원/관광지/일정/커뮤니티 API 명세
- 와이어프레임(메인·탐색·일정편집·커뮤니티·인증)
- 코딩 컨벤션, 백/프론트 로컬 환경 세팅

3단계 — 개발 (Week 3~5)

- 회원: Member·JWT·Spring Security, 회원가입/로그인/로그아웃/수정/탈퇴, 헤더 상태 반영
- 관광지: TourAPI 수집 배치, 지역·카테고리·키워드 조회, 상세 슬라이드, 네이버맵 마커
- 즐거찾기: 토글 API, 목록, 도시 첫 추가 시 자동 연동
- 일정: Trip·TripBlock, 드래그앤드롭 타임라인, 후보군 도시 자동분류
- 이동시간: ODSay(대중교통)·T Map(자동차·도보), 모드별 캐시, 경로 폴리라인 시각화

4단계 — 커뮤니티·고도화 (Week 6~)

- 커뮤니티: 공유게시판(작성/목록/상세/수정/삭제), 공지 CRUD, 댓글·대댓글, 좋아요·북마크
- 협업: WebSocket(STOMP) 실시간 동기화, 낙관적 락, 공유 링크(VIEW/EDIT), 협업자 초대
- AI 챗봇: Spring AI + gms, 관광지 컨텍스트 Q&A, 반경 3km 주변 추천
- 마이페이지: 7탭(여행·내정보·내장소·방문지도·내가쓴글·북마크·좋아요), 방문 지도

5단계 — QA·정리

- 통합 E2E 테스트, 이동시간 정확도 검증, 크로스 브라우저
- 버그 수정·반응형 점검, 보안 일괄 점검
- 산출물(설계문서·PPT·AI보고서) 정리

4. 개인별 일정 (도메인 분담)

2인 페어가 도메인을 나누어 병렬 진행하고, 핵심 화면(일정 편집)·협업 기능은 공동 작업했다.

주차	전진	송정기
W1~2	요구사항·유스케이스, API 명세	ERD·스키마, 컨벤션·환경 세팅
W3	관광지 TourAPI 수집·조회	회원·JWT·Spring Security
W4	관광지 탐색 UI·네이버맵	즐거찾기·후보군 자동연동
W5	이동시간(ODSay/T Map)·경로 시각화	일정 드래그앤드롭 타임라인
W6~	AI 챗봇·커뮤니티 게시판	실시간 협업·공유링크·마이페이지

실제 커밋 이력 기준 세부 분담은 팀에서 확정·보정. (Git 작성자·MR 이력으로 추적 가능)

5. 마일스톤

마일스톤	목표	완료 기준	상태
M1 — 설계 완료	Week 2	ERD·API·와이어프레임 확정	✓
M2 — 회원+관광지	Week 4	로그인·관광지 조회·즐거찾기 동작	✓
M3 — 일정 기능	Week 5	드래그 타임라인·이동시간·저장 동작	✓
M4 — MVP+고도화	Week 6	커뮤니티·협업·AI 챗봇·마이페이지 동작	✓
M5 — 제출	막바지	QA·산출물·발표 준비 완료	✓

6. 리스크 & 대응

리스크	대응
TourAPI 할당량 초과	초기 일괄 적재 후 DB 서비스, call limiter
ODsay/T Map 응답 지연·쿼터	좌표 기반 모드별 캐시 + 노선 폴리라인 영구 캐시
드래그앤드롭 구현 복잡도	vuedraggable 활용
실시간 협업 동시 편집 충돌	version 낙관적 락으로 후행 저장 거부·재동기화
2인 일정 지연	주 1회 동기화, 우선순위 재조정

화면 설계서

Vue 3 + Vite · Pinia · Vue Router. IA(정보구조) 단위로 라우트를 구획했다.

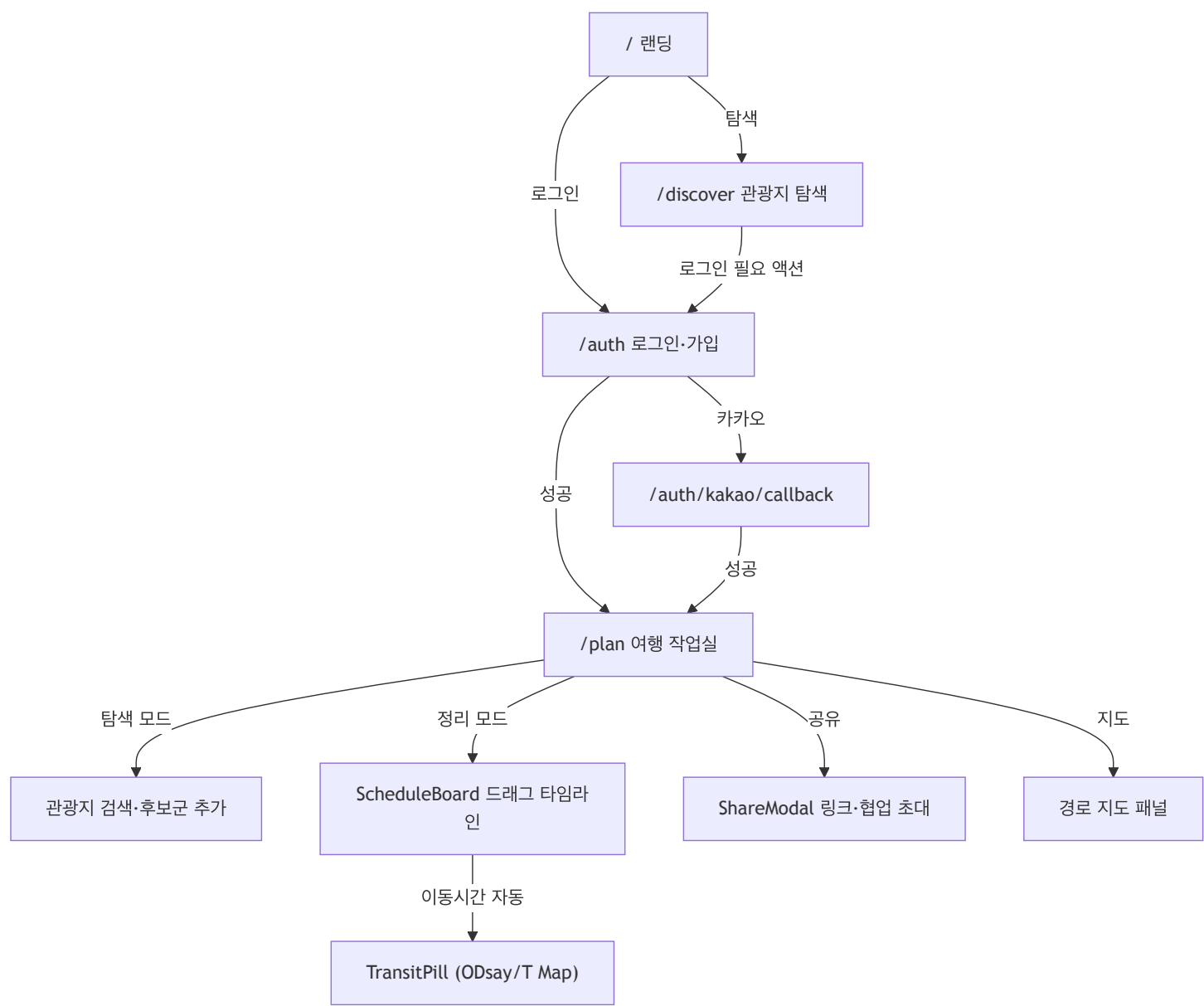
1. 정보 구조(IA) & 라우트

그룹	경로	화면	권한
공개	/	LandingView (히어로·CTA)	비회원
공개	/about	AboutView (소개·기술·팀)	비회원
공개	/discover	ExploreView (관광지 탐색)	비회원 전용(로그인 시 /plan)
인증	/auth	AuthView (로그인/회원가입·카카오)	비회원
인증	/auth/kakao/callback	KakaoCallbackView	비회원
작업실	/plan/:tripId?	PlanView (탐색·정리 통합)	회원(공유링크 예외)
커뮤니티	/community	CommunityView (목록)	비회원 열람
커뮤니티	/community/write , /community/:id/edit	CommunityWriteView	회원
커뮤니티	/community/:id	CommunityPostView (상세)	비회원 열람
관리자	/admin	AdminView (TourAPI 동기화)	관리자
마이페이지	/mypage + 7탭	MyPageLayout → trips·profile·places·map·posts·bookmarks·likes	회원
오류	/:pathMatch(.*)*	NotFoundView	-

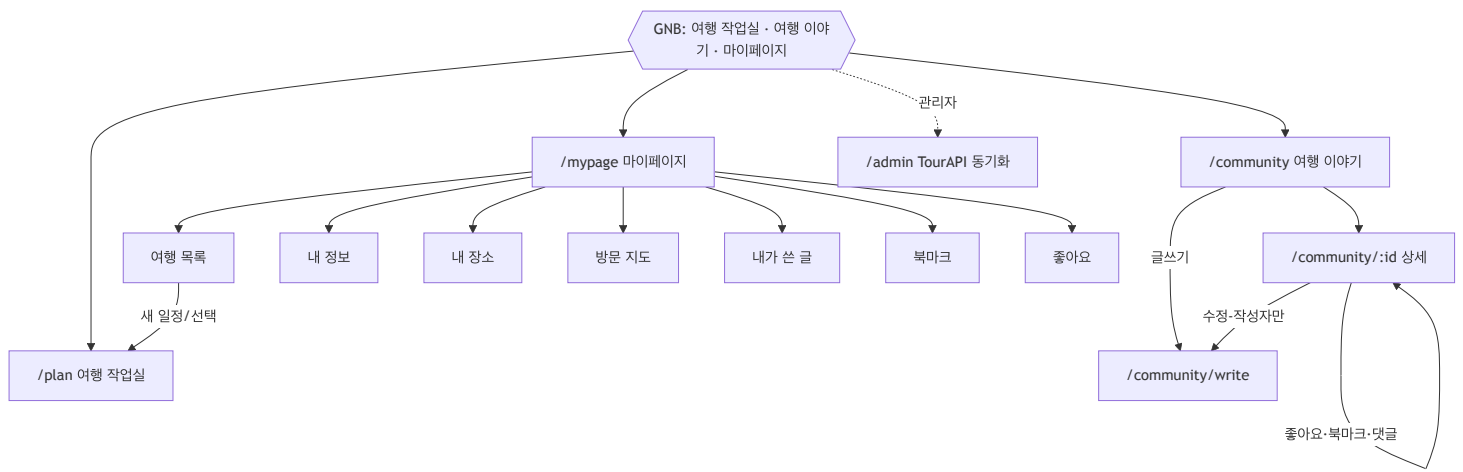
라우트 가드(router.beforeEach): meta.requiresAuth 검사, 로그인 사용자의 /discover → /plan 리다이렉트, 공유 링크(/plan/:id?s=token) 비로그인 조회 허용, requiresAdmin 역할 검사. 구 경로(/explore · /schedule · /trips · /calendar)는 하위호환 리다이렉트.

2. 화면 흐름도

2-1. 진입 · 여행 작업실



2-2. GNB · 커뮤니티 · 마이페이지



3. 주요 화면 명세

화면	핵심 구성	주요 인터랙션
LandingView	히어로·태그라인·기능 소개·CTA	탐색/로그인 진입
ExploreView	네이버맵 + 좌측(검색·지역/시군구/카테고리 필터·목록)	마커 판·InfoWindow, 상세 슬라이드, 후보군 드래그
AuthView	로그인/회원가입 탭, 이메일·비밀번호, 카카오 버튼	폼 검증, OAuth
PlanView (작업실)	헤더(일정 선택·모드 토글·협업 아바타·공유/지도) + 탐색 사이드바/ScheduleBoard	모드 전환, 드래그앤드롭, 실시간 협업 커서
ScheduleBoard	Day 탭 × 시간 그리드, 후보군 사이드바, TransitPill, 지도 패널	블록 드래그·체류시간 핸들·삭제존, 이동수단 드롭다운
CommunityView	정렬(최신/인기)·검색·글쓰기, PostCard 그리드	카드 클릭→상세
CommunityWriteView	제목·커버 업로드·Tiptap 에디터·일정 연결	작성/수정
CommunityPostView	제목·커버·본문·작성자·연결 일정·댓글	좋아요/북마크/댓글, 작성자 수정·삭제
MyPage 7탭	셀(아바타·인사·계정배지) + 탭 라우터뷰	프로필 편집, 내 장소 CRUD, 방문 지도
AdminView	TourAPI 전체/부분 동기화 컨트롤	지역·콘텐츠타입 선택 동기화

4. 디자인 시스템 요약

토큰	값
주요 색상	purple(주조)·teal·pink·blue·amber + grays
타이포	heading / body / label / caption 스케일
간격	4px ~ 28px 스텝
컴포넌트	Button·Input·Card·Modal·Chip·Tab, GNB(3탭)·사이드바·푸터
반응형	데스크톱(1280px+) 우선, 태블릿(768px) 지원
상태 처리	로딩(스피너/스켈레톤)·빈 상태(EmptyState)·토스트(AppToast)

상세 토큰·프레임은 docs/02_design/tripcraft_figma_spec_v0.2.md , UX 재설계 의사결정은 ux_replan_v2.md 참조.

5. 상태 관리(Pinia)

스토어	역할
auth	사용자·로그인 상태, login/signup/kakaoLogin/logout/fetchMe
activeTrip	현재 활성 일정 id (localStorage 유지)
collab	실시간 협업 참가자·grab lock·색상·WebSocket(STOMP) 연결
attractionChat	관광지별 챗봇 세션(messages-conversationId)·아코디언 상태
toast	전역 토스트 메시지

6. 와이어프레임 (핵심 5종)

중간 충실도(mid-fidelity) 목업. 색상은 디자인 시스템 토כן(purple #534AB7 , teal #0F6E56 , pink #993556 , blue #185FA5)을 따른다.
ID 표기는 tripcraft_figma_spec_v0.2.md 의 화면 분류(M/A/B/C/D)와 일치.

화면 M — 일정 생성 모달 (ScheduleModal)

여행 제목·기간(시작/종료일)·인원을 입력해 새 일정을 생성하는 모달.

«화면 M» 일정 생성 모달 — ScheduleModal

새 여행 일정 만들기

여행 제목

예) 제주 3박4일

여행 기간

시작일

종료일

인원

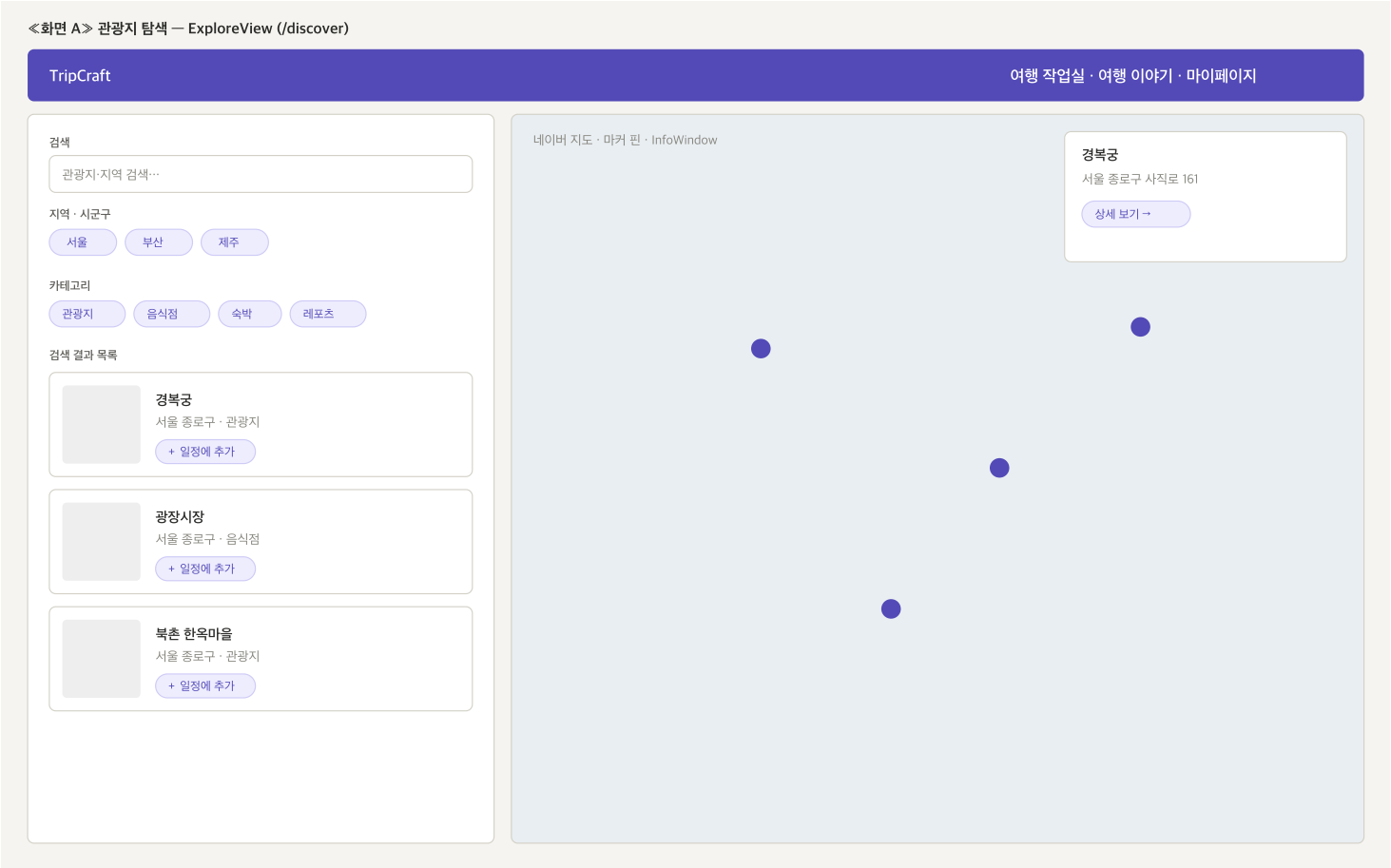
- 2명 +

취소

만들기

화면 A — 관광지 탐색 (ExploreView · /discover)

좌측 검색·지역/시군구/카테고리 필터·결과 목록, 우측 네이버 지도(마커 핀·InfoWindow). 카드의 「+ 일정에 추가」로 후보군 등록.



화면 B — 내 일정 작업실 (PlanView / ScheduleBoard)

상단 톨바(일정 선택·협업 아바타·공유/협업자), 좌측 후보군(도시별·드래그 소스·삭제존), 중앙 Day × 시간 그리드(확정 블록 + TransitPill), 우측 경로 지도 패널. 다른 협업자의 실시간 커서가 함께 표시된다.

<화면 B> 내 일정 작업실 — PlanView / ScheduleBoard (드래그앤드롭 + 실시간 협업)

제주 3박4일 ▾

3박4일 · 2명

협업 중

공유하기

협업자

후보군 (도시별)

▾ 제주시

성산일출봉

관광지 · 드래그하여 배치

흑돼지 거리

음식점

제주 호텔

숙박

▾ 사귀포시

천지연폭포

관광지

여기로 끌어 삭제

Day 1 (월)

Day 2

Day 3

Day 4

09

09:00 성산일출봉

체류 90분

11

대중교통 25분 ▾

13

12:30 흑돼지 거리

체류 60분

자동차 18분 ▾

15

15:00 제주 호텔 (이동 중)

17

19

전진

경로 지도 패널 — 구간·도보·역마커

화면 C — 커뮤니티 (CommunityView)

정렬(최신/인기)-검색-글쓰기, PostCard 그리드, 우측 공지-인기 게시물 사이드바.

◀화면 C▶ 커뮤니티 — CommunityView (여행 이야기)

TripCraft

여행 작업실 · 여행 이야기 · 마이페이지

최신순

인기순

게시글 검색...

글쓰기

제주 가족여행 후기

홍길동 · 2026.06.20

♥ 24 💬 8 🏷️

부산 2박3일 코스

김여행 · 2026.06.18

♥ 51 💬 14 🏷️

서울 맛집 투어

이맛집 · 2026.06.15

♥ 33 💬 5 🏷️

강릉 바다 여행

박파도 · 2026.06.12

♥ 19 💬 3 🏷️

경주 역사 탐방

최신라 · 2026.06.10

♥ 42 💬 11 🏷️

전주 한옥 1박

정한옥 · 2026.06.08

♥ 28 💬 6 🏷️

공지사항

· 서비스 업데이트 안내

· 커뮤니티 이용 규칙

· 신규 지역 데이터 추가

🔥 인기 게시물

1. 부산 2박3일 코스

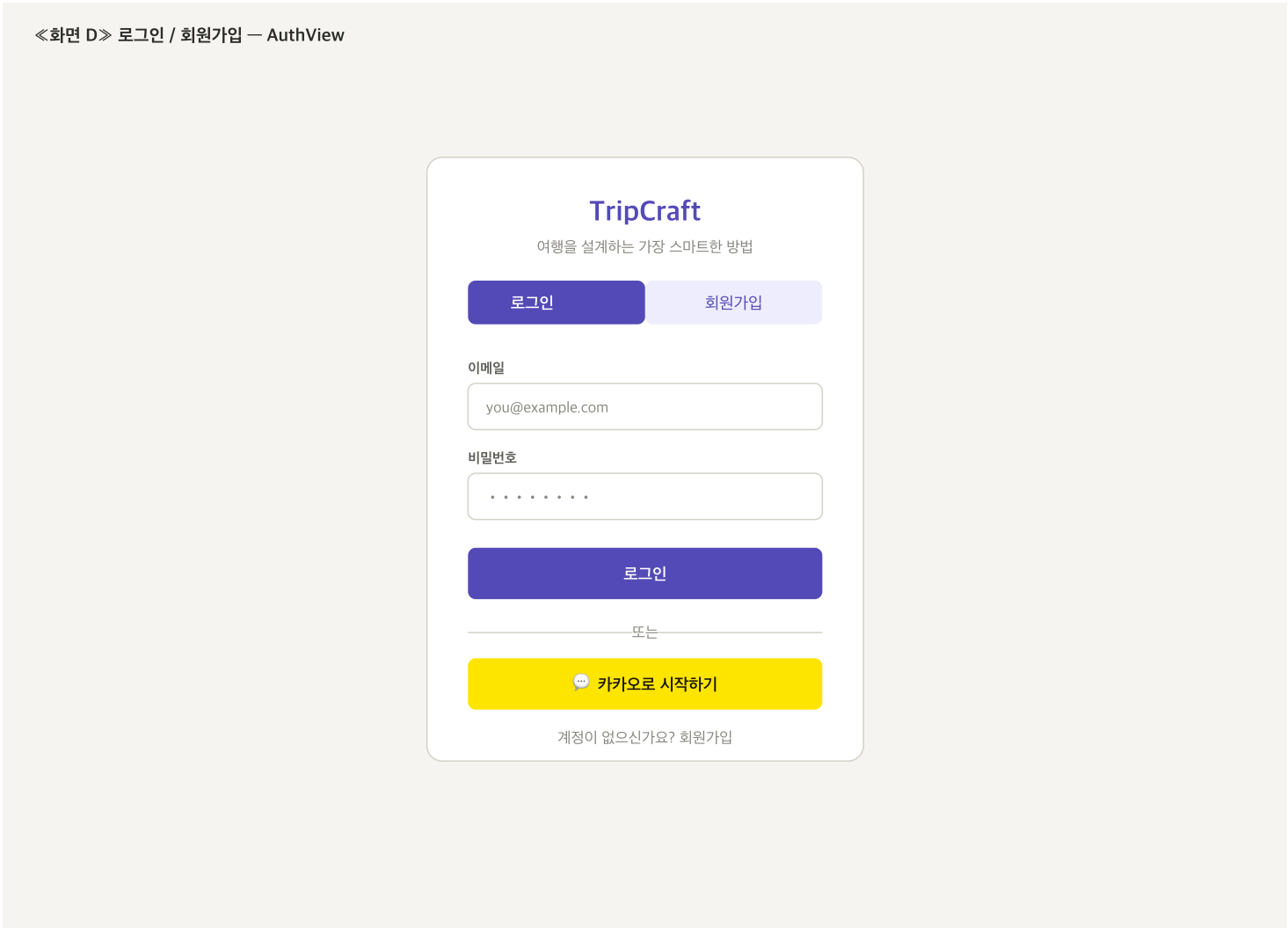
2. 경주 역사 탐방

3. 서울 맛집 투어

4. 제주 가족여행 후기

화면 D — 로그인 / 회원가입 (AuthView)

로그인/회원가입 탭, 이메일·비밀번호 폼, 카카오 시작 버튼.



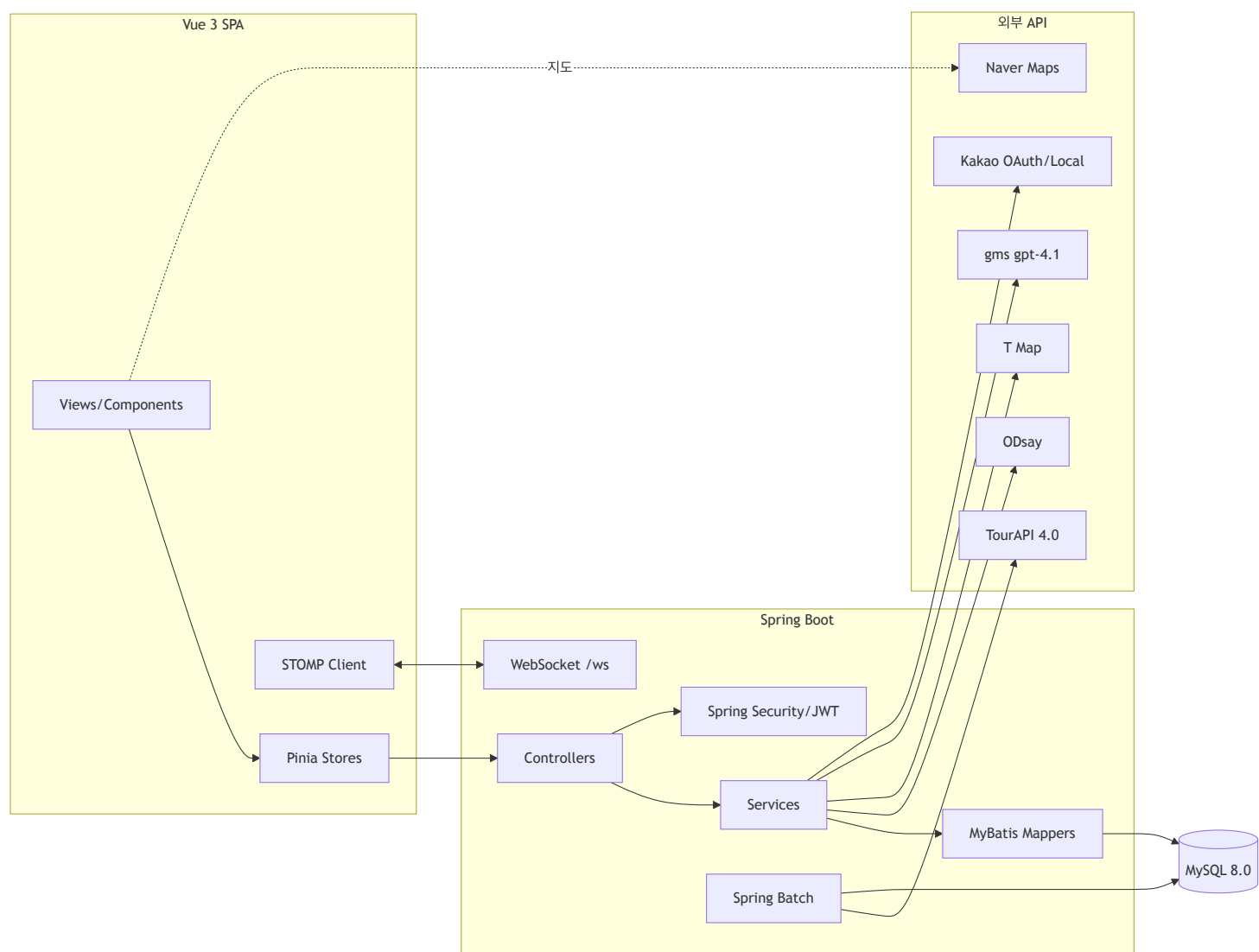
기타 참고 문서

기술 스택 · 시스템 구조도 · 외부 API 연동 · 코딩 컨벤션 · 핵심 알고리즘을 한 문서로 정리.

1. 기술 스택

Layer	기술
Backend	Java 21 · Spring Boot 3.5.0 · Spring Security · Spring Batch · Spring AI · MyBatis · MySQL 8.0 · Gradle(Kotlin DSL)
Frontend	Vue.js 3 + Vite · Pinia · Vue Router · vuedraggable · Tiptap(에디터)
인증	JWT (Access 30분 + Refresh 7일) · BCrypt · Kakao OAuth
실시간	Spring WebSocket + STOMP + SockJS
외부 API	한국관광공사 TourAPI 4.0 · ODSay · T Map(SKT) · Naver Maps · Kakao Local · gms(OpenAI 호환 프록시, gpt-4.1)

2. 시스템 구조도



- **인증 흐름:** 로그인 → access_token · refresh_token 을 **HttpOnly 쿠키**로 발급 → JwtAuthenticationFilter 가 매 요청 access_token 쿠키 검증 → 만료 시 /api/auth/refresh (refresh 쿠키)로 재발급.
- **공통 응답:** 모든 API는 ApiResponse<T> = { success, data, message, errorCode } 로 래핑.
- **권한:** 서버 측 소유권 검증(클라이언트 권한만으로 처리 금지). 관리자 기능은 role=ADMIN 검사.

3. 외부 API 연동

API	클라이언트/설정	용도	최적화
TourAPI 4.0	TourApiClient , TourApiSyncJobConfig , TourApiCallLimiter	관광지 master+상세 수집 (areaBasedList2·detailCommon2 등)	초기 일괄 배치 → 자체 DB 서비스, api_modified_at 증분 동기화, 호출 제한
ODsay	OdsayClient	대중교통 경로(searchPubTransPathT)· 노선 폴리라인(loadLane)	좌표 route_key + 모드별 transit_cache , 노선은 lane_polyline 영구 캐시
T Map(SKT)	TMapClient	자동차·도보 경로, 택시 예상요금, GeoJSON 좌표	transit_cache.route_coords/taxi_fare 캐시
Naver Maps	프론트 SDK	마커·InfoWindow·경로 폴리라인 시각화	-
Kakao	KakaoLocalClient	OAuth 로그인 + Local 키워드 검색 (커스텀 장소)	-
gms (Spring AI)	ChatClientConfig	관광지 AI 챗봇(gpt-4.1)	InMemory ChatMemory 멀티턴, 회원 전용

모든 API 키는 환경변수로 주입(TOUR_API_KEY · ODSAY_API_KEY · TMAP_API_KEY · KAKAO_* · GMS_KEY) — 소스 하드코딩 금지.

4. 코딩 컨벤션 (요약)

- **Git**: GitLab MR 기반. master ← feature/{도메인-기능} · fix/* · docs/* (MR로 머지). master 직접 push 금지, 1회 이상 리뷰 후 머지.
- **커밋**: Conventional Commits — feat(plan): 드래그 블록 시간표 추가 . 1커밋=1논리단위.
- **백엔드**: 도메인 중심 패키지, 클래스 PascalCase·메서드 camelCase·상수 UPPER_SNAKE·DB snake_case·URL kebab-case. REST 복수명사, 동사 URL 금지.
- **MyBatis**: #{ } 바인딩만 사용 (\${ } 금지 — SQL Injection 방지).
- **프론트**: 컴포넌트 PascalCase, 뷰는 ~View.vue , composable useXxx , <script setup> 순서 규약.
- **DB**: PK id BIGINT AUTO_INCREMENT , created_at/updated_at , 논리삭제 deleted_at , FK 참조단수_id .

정본: docs/03_dev/conventions.md .

5. 핵심 알고리즘 & 적용 패턴

5.1 이동 시간 자동 계산 & 캐싱

- 블록 배치/이동/삭제 시 인접 두 장소 간 이동시간을 계산해 TransitPill 로 삽입.
- 캐시 키 = (좌표 route_key, departure_hour, request_mode) → 모드별 독립 캐시(transit_cache).
- 캐시 정밀도는 system_config 로 1~5단계 조정(시간무시 ~ 시간별). 러시아워 경계도 설정값으로 분기.
- 캐시 미스 시에만 ODSay/T Map 호출 → 무료 플랜 쿼터 보호. 노선 형상은 lane_polyline 영구 캐시.

5.2 드래그앤드롭 타임라인

- vuedraggable 기반. 후보군 → Day 타임라인 드롭 시 trip_block 생성, 체류시간 핸들로 30분 스넵.
- 같은 장소 중복 방문 허용, 사이드바 드롭으로 삭제. display_order 로 날짜 내 순서 관리.
- 날짜 범위 이탈은 DB TRIGGER로 방어(trg_trip_block_date_insert/update).

5.3 실시간 협업 (낙관적 락)

- WebSocket(STOMP) /topic/trip/{id} 로 변경 브로드캐스트, presence로 참가자·커서 동기화.
- 동시 편집 충돌은 trip_block.version 낙관적 락 — 사용자 편집 시 +1, 후행 저장은 version 불일치로 거부 후 재동기화. (transit 재계산은 version 미변경)

5.4 AI 주변 추천

- 관광지 좌표 기준 반경 ~3km·최대 8곳을 ST_Distance_Sphere 로 거리순 조회 → 시스템 프롬프트 컨텍스트 주입.
- gms gpt-4.1 멀티턴 응답에서 실제 언급된 장소만 버튼화 → 지도 판·상세 이동, 뒤로가기 시 대화·스크롤 복원.

5.5 데이터 보존 정책 (삭제 전략)

- 작성자/일정 삭제 시 게시글·공지 **SET NULL** 보존("탈퇴한 사용자").
- 후보군→블록 **RESTRICT**로 모달 확인 UX. 글 **소프트 딜리트**(deleted_at) + 북마크 보존.

API 명세서

REST 약 72개 + WebSocket(STOMP) 실시간 협업 채널.

1. 개요

1-1. 공통 응답 형식

모든 REST 응답은 ApiResponse<T> 래퍼로 감싼다. (JsonInclude.NON_NULL — null 필드는 직렬화 생략)

```
{ "success": true, "data": { }, "message": null, "errorCode": null }
```

필드	타입	설명
success	boolean	성공 여부
data	T null	성공 시 페이로드 (없으면 생략)
message	string null	실패 시 사용자 메시지
errorCode	string null	실패 시 에러 코드

1-2. 인증 방식

- **JWT (쿠키 기반)**. 로그인 성공 시 서버가 access_token · refresh_token 을 **HttpOnly 쿠키**로 발급.
- 보호된 요청: JwtAuthenticationFilter 가 access_token 쿠키를 검증 → memberId · role 을 SecurityContext에 적재.
- 토큰 만료 시: POST /api/auth/refresh (refresh_token 쿠키 사용)로 재발급.
- 컨트롤러는 @AuthenticationPrincipal Long memberId 로 현재 회원 식별.
- CORS: allowCredentials=true , 메서드 GET/POST/PUT/DELETE/PATCH/OPTIONS 허용.

1-3. 권한 레벨 (SecurityConfig 기준)

레벨	의미
공개	비로그인 호출 가능 (permitAll)
인증	로그인 필요 (anyRequest authenticated)
인증*	보안상 공개 경로지만 memberId 가 있어야 의미 있는 응답(내 데이터 조회)
ADMIN	ROLE_ADMIN 필요

공개 경로: POST /api/auth/{login,signup,refresh,logout,kakao} , GET /api/attractions/** ,
GET /api/posts/** , GET /api/notices/** , GET /api/trips/*/blocks-summary , GET /api/trips/shared/** ,
GET /uploads/** , /ws/** , Swagger 경로.

ADMIN 전용: /api/admin/** , POST·PUT·DELETE /api/notices/** .

2. 엔드포인트 카탈로그

2-1. 인증 (/api/auth)

Method	Path	권한	설명	요청
POST	/api/auth/signup	공개	회원가입	SignupRequest
POST	/api/auth/login	공개	로그인(쿠키 발급)	LoginRequest
POST	/api/auth/kakao	공개	카카오 소셜 로그인	{ code }
POST	/api/auth/refresh	공개	엑세스 토큰 재발급	refresh_token 쿠키
POST	/api/auth/logout	공개	로그아웃(토큰 무효화)	refresh_token 쿠키
GET	/api/auth/me	인증	내 정보 조회	—

2-2. 회원 (/api/members)

Method	Path	권한	설명	요청
PATCH	/api/members/me/nickname	인증	닉네임 변경(중복 검사)	UpdateNicknameRequest
PATCH	/api/members/me/password	인증	비밀번호 변경(현재 PW 확인)	UpdatePasswordRequest
GET	/api/members/me/profile-image	인증	프로필 이미지 URL 조회	—
POST	/api/members/me/profile-image	인증	프로필 이미지 업로드(교체)	multipart: file
DELETE	/api/members/me/profile-image	인증	프로필 이미지 삭제	—
GET	/api/members/me/visited-regions	인증	방문 시도 코드 목록(내 지도)	—
GET	/api/members/search?q=	인증	협업자 초대용 회원 검색(닉네임·이메일)	query q
DELETE	/api/members/me	인증	회원 탈퇴(비밀번호 확인, 하드 딜리트)	WithdrawRequest

2-3. 관광지 (/api/attractions)

Method	Path	권한	설명	요청
GET	/api/attractions/regions	공개	시도+시군구 트리 조회	—
GET	/api/attractions	공개	관광지 검색(페이지)	query: keyword, region, sigungu, category, page, size
GET	/api/attractions/{id}	공개	관광지 상세(detail* 포함)	path id

2-4. 관광지 AI 챗봇 (/api/attractions)

Method	Path	권한	설명	요청
POST	/api/attractions/{id}/chat	인증	관광지 컨텍스트 멀티턴 Q&A(Spring AI)	AttractionChatRequest{ message, conversationId }

2-5. 장소 검색 / 내 장소 (/api/places , /api/my-places)

Method	Path	권한	설명	요청
GET	/api/places/search?query=	인증	Kakao Local 키워드 장소 검색	query query
GET	/api/my-places	인증	내 커스텀 장소 목록	—
POST	/api/my-places	인증	내 커스텀 장소 등록	MemberPlaceRequest
DELETE	/api/my-places/{id}	인증	내 커스텀 장소 삭제	path id

2-6. 여행 일정 (/api/trips)

Method	Path	권한	설명	요청
GET	/api/trips	인증	내 일정 목록	—
GET	/api/trips/collaborating	인증	내가 협업자인 일정 목록	—
POST	/api/trips	인증	일정 생성	TripCreateRequest
GET	/api/trips/{id}	인증	일정 상세(소유자·협업자)	path id
DELETE	/api/trips/{id}	인증	일정 삭제	path id
PUT	/api/trips/{id}/share	인증	공유 링크 접근레벨 설정 → {access, token}	{ access }
GET	/api/trips/shared/{token}	공개	공유 토큰으로 일정 조회	path token
POST	/api/trips/{tripId}/copy	인증	공유 일정 내 일정으로 복제(날짜 재계산)	TripCopyRequest
PATCH	/api/trips/{tripId}/default-transit-mode	인증	기본 이동수단 변경	{ mode }
GET	/api/trips/{id}/blocks-summary	공개	블록 요약(공유 미리보기)	path id

협업자

Method	Path	권한	설명	요청
GET	/api/trips/{id}/collaborators	인증	협업자 목록	path id
POST	/api/trips/{id}/collaborators	인증	협업자 초대	{ memberId, role=EDITOR VIEWER }
DELETE	/api/trips/{id}/collaborators/{targetMemberId}	인증	협업자 제거	path

후보군(보관함)

Method	Path	권한	설명	요청
POST	/api/trips/{id}/candidates	인증	관광지 후보 추가	CandidateAddRequest{ attractionId }
POST	/api/trips/{id}/candidates/custom	인증	커스텀 장소 후보 추가	CustomCandidateRequest
POST	/api/trips/{id}/candidates/from-place/{placeId}	인증	내 장소에서 후보 추가	path
DELETE	/api/trips/{id}/candidates/{candidateId}	인증	후보 삭제 (블록 있으면 RESTRICT)	path

타임라인 블록

Method	Path	권한	설명	요청
POST	/api/trips/{id}/blocks	인증	블록 배치(이동시간 자동 계산)	BlockCreateRequest
PUT	/api/trips/{id}/blocks/{blockId}	인증	블록 수정(시간·순서·메모, 낙관적 락)	BlockUpdateRequest
DELETE	/api/trips/{id}/blocks/{blockId}	인증	블록 삭제	path

2-7. 이동 시간 (/api/transit) — ODsay-T Map

Method	Path	권한	설명	요청(query)
GET	/api/transit	인증	두 관광지 간 이동시간	fromId,toId,hour,mode
GET	/api/transit/by-coords	인증	좌표 기반 이동시간 (커스텀)	fromLat,fromLng,toLat,toLng,hour,mode
GET	/api/transit/detail	인증	대중교통 경로 단계 상세	fromId,toId,hour
GET	/api/transit/by-coords/detail	인증	좌표 기반 경로 단계 상세	좌표 4 + hour
GET	/api/transit/driving-options	인증	자동차 단일 옵션 조회	fromId,toId,hour,optionIndex
GET	/api/transit/by-coords/driving-options	인증	좌표 기반 자동차 옵션	좌표 4 + hour,optionIndex
POST	/api/transit/select	인증	대중교통 경로 선택 (pathIndex)	fromId,toId,hour,pathIndex
POST	/api/transit/select-driving	인증	자동차 옵션 적용	fromId,toId,hour,optionIndex
GET	/api/transit/route-segments	인증	경로 구간(색·도보· 역마커)	fromId,toId,hour
GET	/api/transit/by-coords/route-segments	인증	좌표 기반 경로 구간	좌표 4 + hour
GET	/api/transit/walking-coords	인증	도보 경로 좌표	startLat,startLng,endLat,endLng

mode = PUBLIC_TRANSIT (기본) · DRIVING · WALKING . hour 기본 9(출발 시각, 캐시 키).

2-8. 커뮤니티 게시물 (/api/posts)

Method	Path	권한	설명	요청
GET	/api/posts	공개	게시글 목록(정렬·검색·페이지)	query: page,size,sort=latest popular,keyword
GET	/api/posts/me	인증*	내 게시물 목록	query: page,size
POST	/api/posts	인증	일정 공유 게시물 작성	PostCreateRequest
GET	/api/posts/{id}	공개	게시글 상세(공유 일정 뷰어 포함)	path id
PATCH	/api/posts/{id}	인증	게시글 수정	PostUpdateRequest
DELETE	/api/posts/{id}	인증	게시글 삭제(소프트 딜리트)	path id
POST	/api/posts/{id}/likes	인증	좋아요 토글	path id

2-9. 댓글 (/api/posts/{postId}/comments)

Method	Path	권한	설명	요청
GET	/api/posts/{postId}/comments	공개	댓글·대댓글 목록	path postId
POST	/api/posts/{postId}/comments	인증	댓글/대댓글 등록	CommentCreateRequest{ content, parentId? }
DELETE	/api/posts/{postId}/comments/{commentId}	인증	댓글 삭제 (본인/ADMIN)	path

2-10. 좋아요·북마크 (/api/likes , /api/posts , /api/bookmarks)

Method	Path	권한	설명	요청
GET	/api/likes/me	인증*	내가 좋아요한 글 목록	query: page,size
POST	/api/posts/{postId}/bookmark	인증	북마크 토글	path postId
GET	/api/bookmarks/me	인증*	내 북마크 목록(삭제글 표시)	query: page,size

2-11. 공지사항 (/api/notices)

Method	Path	권한	설명	요청
GET	/api/notices	공개	최신 공지 5건(사이드바)	—

공지 작성/수정/삭제(POST·PUT·DELETE /api/notices/**)는 ADMIN 전용으로 보안 설정됨.

2-12. 이미지 (/api/images)

Method	Path	권한	설명	요청
POST	/api/images/upload	인증	게시글 이미지/대표사진 업로드(draft)	multipart: file , query type=cover?
DELETE	/api/images/cover-draft	인증	임시 대표사진 정리	—

2-13. 관리자 — 관광지 동기화 (/api/admin/attractions)

Method	Path	권한	설명	요청
POST	/api/admin/attractions/sync/regions	ADMIN	시도·시군구 참조 동기화(TourAPI)	—
POST	/api/admin/attractions/sync	ADMIN	전체 관광지 수집(수 분 소요)	—
POST	/api/admin/attractions/sync/partial	ADMIN	부분 수집(테스트)	query: areaCode,contentTypeId

3. WebSocket 실시간 협업 (STOMP)

WebSocketConfig 기준. 일정 공동 편집의 커서/아바타·블록 변경 실시간 동기화.

항목	값
핸드셰이크 엔드포인트	/ws (SockJS) — JWT 핸드셰이크 인터셉터 인증
앱(발행) prefix	/app
브로커(구독) prefix	/topic , /queue
발행 — 커서/프레즌스	SEND /app/trip/{tripId}/pointer
구독 — 프레즌스 업데이트	SUBSCRIBE /topic/trip/{tripId}/presence
구독 — 일정 변경 이벤트	SUBSCRIBE /topic/trip/{tripId} (블록 추가·수정·삭제 등 TripEvent , seq 포함)

- 권한: JwtChannelInterceptor 가 SUBSCRIBE 시 조회 권한, SEND 시 편집 권한을 검증(세션 캐시).
- 프레즌스: 좌표는 절대 픽셀이 아닌 zone 기반 비율 좌표. stale(기본 5s) 경과 시 evict. leave / keepalive 페이로드 지원.

4. 대표 요청/응답 예시

4-1. 회원가입 — POST /api/auth/signup

```
// Request
{ "email": "user@example.com", "password": "password123", "nickname": "여행자" }
// Response 201
{ "success": true }
```

4-2. 로그인 — POST /api/auth/login

```
// Request
{ "email": "user@example.com", "password": "password123" }
// Response 200 — Set-Cookie: access_token, refresh_token (HttpOnly)
{ "success": true }
```

4-3. 관광지 검색 — GET /api/attractions?region=1&category=12&page=0&size=20

```
// Response 200 (AttractionPageResponse - 요약)
{ "success": true, "data": {
  "content": [ { "id": 101, "title": "경복궁", "contentType": 12,
    "addr1": "서울 종로구 ...", "firstImage": "https://...",
    "latitude": 37.5796, "longitude": 126.9770, "favorite": false } ],
  "page": 0, "size": 20, "totalElements": 134, "totalPages": 7 } }
```

4-4. 일정 생성 — POST /api/trips

```
// Request (TripCreateRequest)
{ "title": "부산 2박3일", "startDate": "2026-07-01", "endDate": "2026-07-03",
  "memberCount": 2, "defaultTransitMode": "PUBLIC_TRANSIT" }
// Response 201 - 생성된 tripId
{ "success": true, "data": 42 }
```

4-5. 블록 배치 — POST /api/trips/42/blocks

```
// Request (BlockCreateRequest)
{ "candidateId": 555, "tripDate": "2026-07-01", "startTime": "10:00",
  "durationMinutes": 90, "displayOrder": 1 }
// Response 201 - 생성된 blockId
{ "success": true, "data": 777 }
```

4-6. 이동시간 — GET /api/transit?fromId=101&toId=102&hour=9&mode=PUBLIC_TRANSIT

```
// Response 200 (TransitResponse)
{ "success": true, "data": {
  "durationMinutes": 38, "transportMode": "SUBWAY,BUS", "transferCount": 1,
  "fare": 1500, "totalWalkM": 420, "totalDistanceM": 9200, "label": "추천" } }
```

4-7. 게시글 작성 — POST /api/posts

```
// Request (PostCreateRequest)
{ "title": "부산 다녀왔어요", "content": "...", "tripId": 42 }
// Response 201
{ "success": true, "data": 88 }
```

5. 공통 에러 응답

GlobalExceptionHandler (@RestControllerAdvice)가 예외를 ApiResponse.fail(message, errorCode) 로 변환.

errorCode 규칙

- ResponseStatusException → errorCode = **HTTP 상태 문자열**(예: "401 UNAUTHORIZED" , "409 CONFLICT"), message = 던질 때의 reason.
- DataIntegrityViolationException (DB UNIQUE 위반 등) → HTTP **409**, errorCode = "CONFLICT" , 메시지는 위반 키별 안내.

- 그 외 미처리 예외 → HTTP **500**, `errorCode` = "INTERNAL_SERVER_ERROR" .

```
// 401 - 인증 필요 (ResponseStatusException)
{ "success": false, "message": "로그인이 필요합니다.", "errorCode": "401 UNAUTHORIZED" }
// 409 - 보관함 중복 (DataIntegrityViolation: uq_candidate_place)
{ "success": false, "message": "이미 보관함에 있는 장소예요.", "errorCode": "CONFLICT" }
// 500 - 서버 오류
{ "success": false, "message": "서버 오류가 발생했습니다.", "errorCode": "INTERNAL_SERVER_ERROR" }
```

상황	HTTP	errorCode	비고
인증 누락/만료	401	401 UNAUTHORIZED	AuthEntryPoint
권한 부족(ADMIN 등)	403	403 FORBIDDEN	AccessDeniedHandlerImpl
검증 실패(@Valid)	400	400 BAD_REQUEST	필드 메시지
닉네임 중복 등 비즈니스 충돌	409	409 CONFLICT	ResponseStatusException(CONFLICT)
DB 무결성 위반(보관함·내 장소 중복)	409	CONFLICT	키별 안내 메시지
미처리 예외	500	INTERNAL_SERVER_ERROR	

기술 심화 — 경로 최적화 & 지도 시각화

여행 일정의 두 장소 사이 "이동수단별 경로"를 **자동으로 계산**하고 그 경로를 **지도에 실제처럼 시각화**하는 기능.
프로젝트에서 기술적으로 가장 복잡했던 작업이다. 핵심은 (1) 한 API로 부족한 경로를 **여러 호출로 합성**하고,
(2) 무료 API의 **호출 제한(rate limit)** 을 건디는 **다층 캐싱**을 설계한 것이다.

0. 왜 이렇게 만들었나 — 무료 API 조합

"출발지에서 목적지까지 대중교통/자동차로 몇 분"을 구하는 가장 쉬운 길은 **유료 길찾기 API**(Kakao Mobility 길찾기 등)를 한 번 호출하는 것이다. 좌표 두 개를 넘기면 소요시간·경로·폴리라인을 한 응답으로 돌려준다.

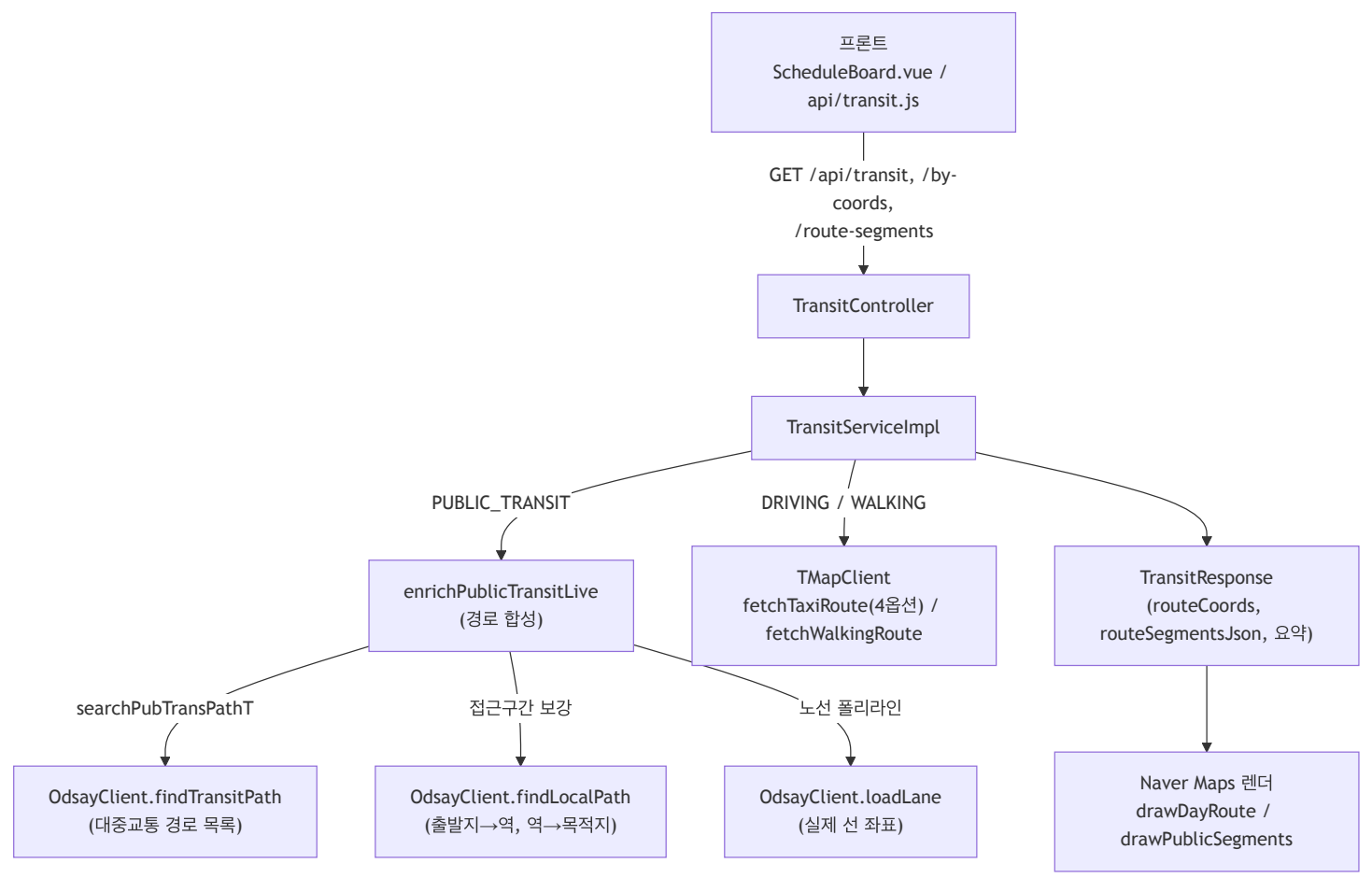
우리는 **비용을 피하려고 무료 API 조합**을 택했다. 대중교통은 ODsay, 자동차·도보는 TMap, 지도 렌더는 Naver Maps를 쓴다. 각 무료 API는 자기 영역만 잘 하므로, 유료 단일 API가 한 번에 주던 결과를 **우리가 직접 합성·파싱**해야 한다.

접근	사용 API	비용	대가(기술 부담)
유료 단일	Kakao Mobility 길찾기	호출당 과금	거의 없음(응답 그대로 사용)
무료 조합(채택)	ODsay(대중교통) + TMap(자동차·도보) + Naver(렌더)	무료	① 응답을 직접 합성·파싱 ② 호출 제한을 캐시 로 방어

즉 "무료"의 대가가 이 보고서의 두 축이다 — **\$2 합성(조립)** 과 **\$3 캐싱**.

1. 전체 그림 (한눈에)

경로 하나에 외부 API 4종이 얹힌다. 프론트가 모드(대중교통/자동차/도보)를 넘기면 백엔드가 해당 API를 호출·합성해 좌표와 요약을 만들고, 프론트가 Naver 지도에 그린다.



어려움의 본질은 세 가지다.

1. **파싱** — Odsay 응답이 깊게 중첩된 JSON(path[] .subPath[] , 교통수단 코드)이라 의미 단위로 풀어야 한다.
2. **조합** — 한 호출로 끝나지 않는다. 도시간(KTX) 경로는 "역→역"만 주므로 접근 구간을 **추가 호출로 합성**하고, 지도 선은 또 **별도 호출**(loadLane)로 그린다.
3. **호출 제한** — 무료 API라 같은 좌표를 짧게 반복 호출하면 **빈 응답**을 준다 → 캐싱이 필수.

2. 외부 API 오케스트레이션 (조립)

① 왜 한 번으로 안 끝나나

ODsay의 도시간 경로(pathType >= 11 , 열차·고속/시외버스)는 **출발역→도착역 구간만** 반환한다. "우리 집 → 서울역", "부산역 → 목적지" 같은 **접근 구간**이 빠져서 총 소요시간이 비현실적으로 짧게 나왔다. (시내 경로 pathType < 11 은 그대로 쓴다.)

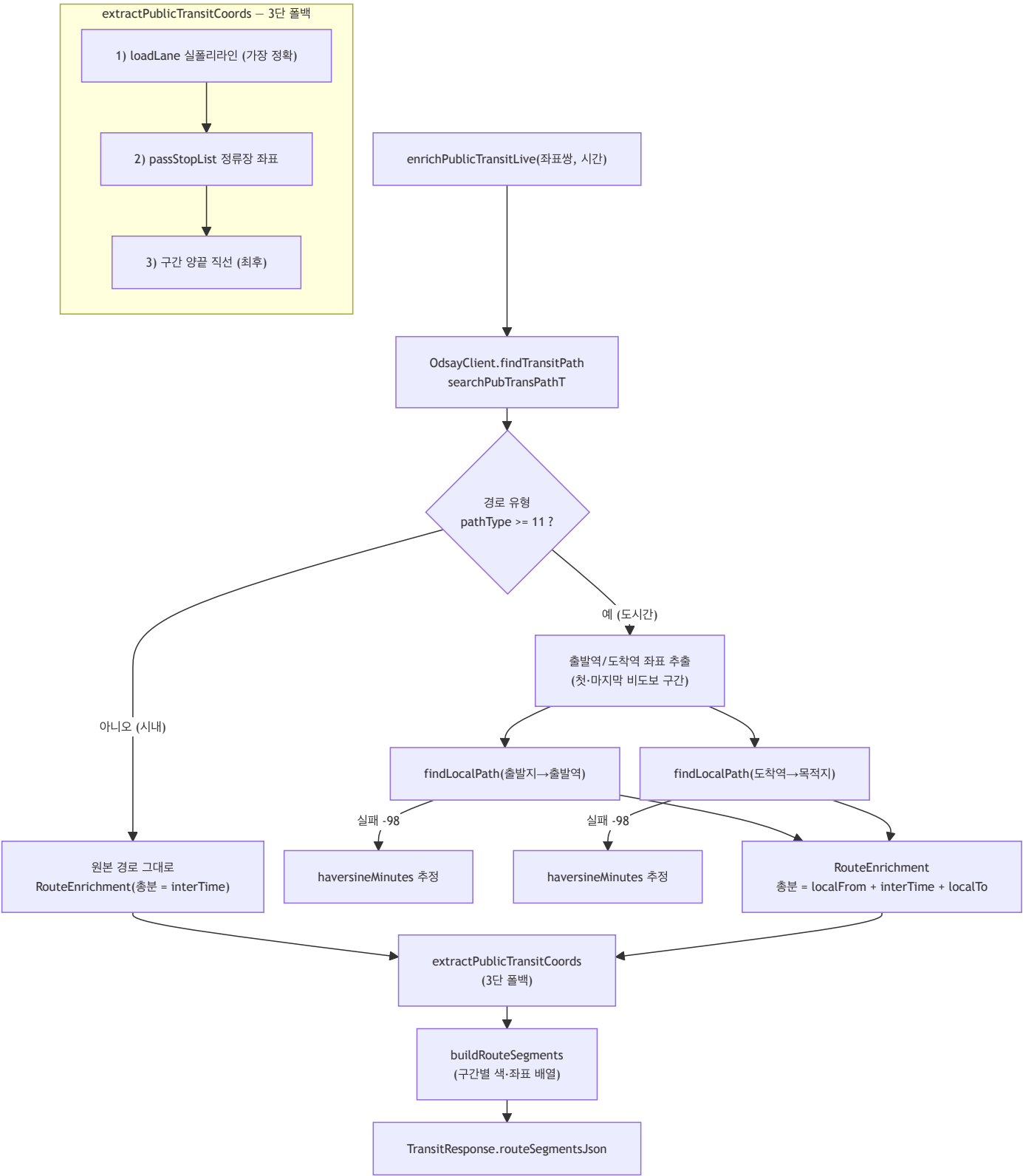
② 어떻게 합성했나

도시간 경로마다 로컬 경로를 추가로 호출해 앞뒤에 붙인다 (`enrichPublicTransitLive`).

```
[출발지] --findLocalPath--> [출발역] ==KTX(원래 응답)== [도착역] --findLocalPath--> [목적지]
    localFrom(예: 46분)           interTime(138분)           localTo(예: 20분)
                                총 = localFrom + interTime + localTo
```

- 비-도보 첫 구간의 시작좌표 = 출발역, 마지막 비-도보 구간의 끝좌표 = 도착역으로 잡고
 `findLocalPath(출발지→출발역)` , `findLocalPath(도착역→목적지)` 를 호출한다.
- 로컬 경로 API가 실패하면(700m 이내 등 -98 코드) **하버사인 거리 기반 추정**(`haversineMinutes` , 평균 30km/h)으로 풀백한다.
- 결과를 `RouteEnrichment`(원본, 총분, `localFrom`, `localTo`) 레코드로 담아 직렬화 → 모달 스텝·지도에서 재사용.

조립 구조를 한 장으로:



자동차(TMap) 는 검색옵션으로 4가지를 각각 호출해 비교 제공한다.

인덱스	searchOption	라벨
0	00	추천
1	02	최단시간
2	01	무료도로
3	10	최소거리

응답에서 주요 도로명(거리순 4개)으로 요약(buildRoadSummary)하고 통행료·택시요금·구간 좌표를 추출한다.

③ 어려움

- **보강 내부의 rate limit.** ODsay가 도시간 경로를 10~20개 반환하면, **경로마다** localFrom/localTo를 2번씩 호출해 한 번의 보강에서 ODsay를 20~40번 두드린다. 그러면 그 안에서 또 rate-limit에 걸려 "한두 경로만 실제 대중교통, 나머지는 추정"으로 망가졌다.
- **경로별 독립 계산 누락(2026-05-29).** 도시간 경로마다 출발역이 다른데(수서 SRT vs 서울역 KTX vs 강남 고속버스), 초기 코드는 첫 경로의 역 좌표로만 localFrom/localTo를 계산해 다른 경로 선택 시 소요시간이 늘어졌다. → 각 경로가 자기 출발역·도착역 좌표로 **독립 계산**하도록 수정.

④ 고민·의사결정

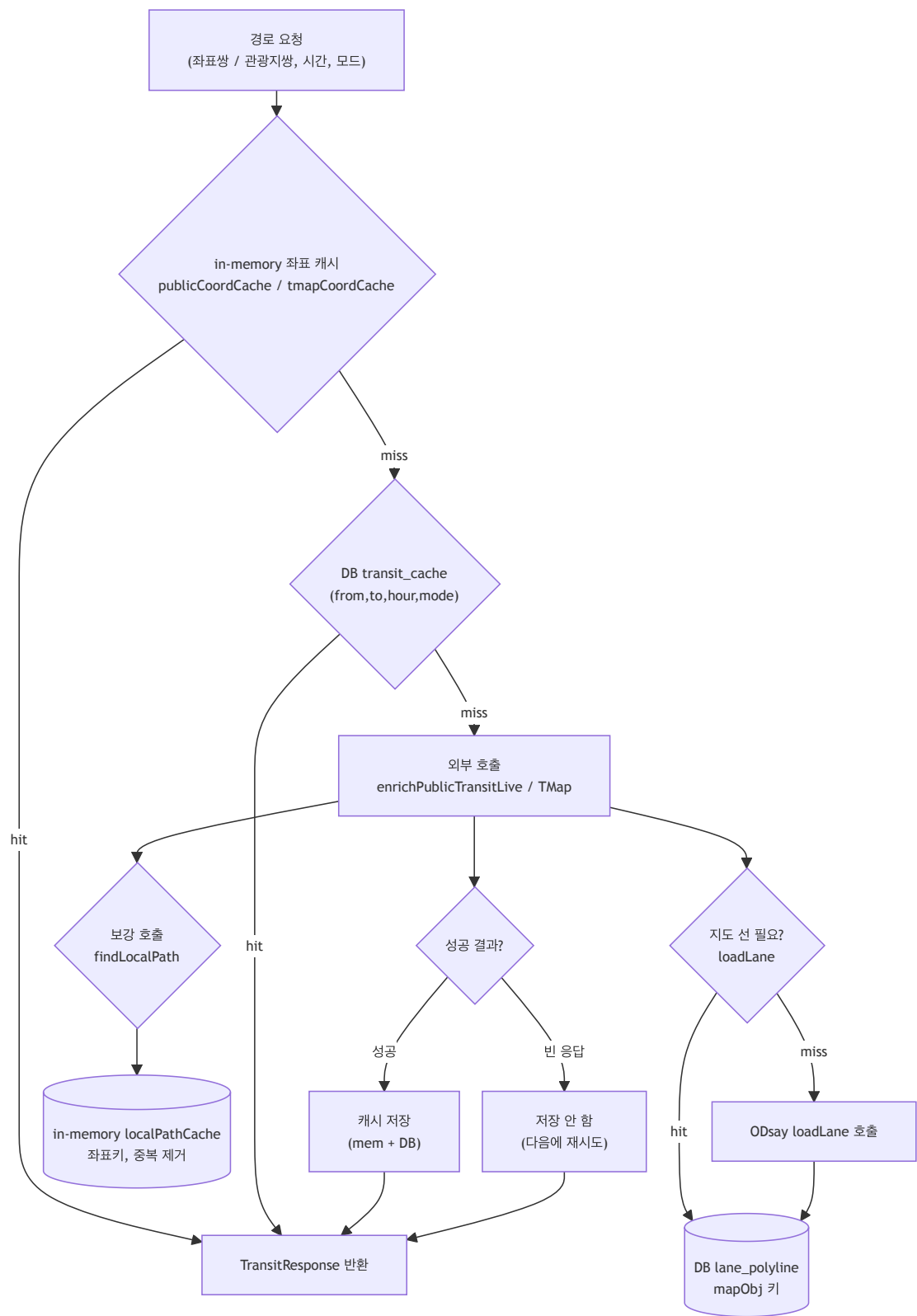
보강이 이 기능에서 **가장 비싼** 부분이다. findLocalPath 를 **좌표키로 캐시 + 중복 제거**했다. 도시간 경로 대부분이 **같은 출발역**(예: 서울역)을 공유하므로, 캐시로 20~40회 호출이 **고유 역 수(보통 1~3회)** 로 줄어든다. 이 결정이 다음 장(캐싱)의 출발점이다.

3. 다층 캐싱 (핵심 설계)

① 왜 캐싱이 필수인가

무료 API는 같은 좌표를 짧은 시간에 반복 호출하면 **빈 응답(rate limit)** 을 준다. 지도를 다시 그리거나 핀 모달을 열 때마다 생호출하면 금방 막힌다. 외부 호출은 느리고 제한이 있으므로, **무엇을 어디에 캐시할지**가 **핵심 설계**였다.

② 어떻게 3계층(DB 2 + in-memory 2)으로 설계했나



계층	저장소	키	대상	비고
① 이동시간 캐시	DB transit_cache	(from_attr, to_attr, hour, request_mode)	관광지쌍 경로·요약 ·path_detail	모드별 행(대중교통 1 + 자동차 4), NONE 도 캐시해 재시도 방지

계층	저장소	키	대상	비고
② 노선 폴리라인	DB lane_polyline	mapObj 키	loadLane 결과	결과 없음도 저장 (불필요 재호출 차단)
③ 좌표 결과	in-memory publicCoordCache · tmapCoordCache	"%.5f,%.5f>%.5f,%.5f" (+mode)	커스텀 장소(by-coords) 경로	세션 캐시, 성공만 저장 (빈 응답 미저장 → 재시도), 1000개 상한
④ 도시내 로컬 경로	in-memory localPathCache (OdsayClient)	좌표키	보강용 findLocalPath 결과	같은 역 공유 중복 제거, 성공·추정만 저장, 2000개 상한

①②는 관광지(attraction id 기반) 경로용으로 처음부터 있었고, ③은 attraction id가 없는 **커스텀 장소** 때문에 추가됐다. **성공 결과만 저장**하는 게 핵심이다.

```
List<RouteEnrichment> cached = publicCoordCache.get(key);
if (cached != null) return cached;
List<RouteEnrichment> fresh = enrichPublicTransitLive(...);
if (!fresh.isEmpty()) publicCoordCache.put(key, fresh);    // 성공만 캐시
return fresh;
```

③ 어려움

커스텀 장소 경로가 "가끔" 모달에 안 뜨고 지도엔 직선만 그려지는 현상이 있었다. 재현 실험:

```
by-coords/detail 같은 좌표쌍 8연타
→ try1~5: 정상(경로 있음)
→ try6~8: EMPTY (빈 응답)
```

원인은 **무캐시**였다. 지도 draw-핀 모달 open마다 Odsay/TMap을 생호출 → 같은 좌표를 짧게 5회 넘기면 Odsay가 빈 응답을 준다. 섞인 날엔 관광지 loadLane 호출까지 같이 굼어 관광지 지도도 깨졌다.

④ 고민·의사결정

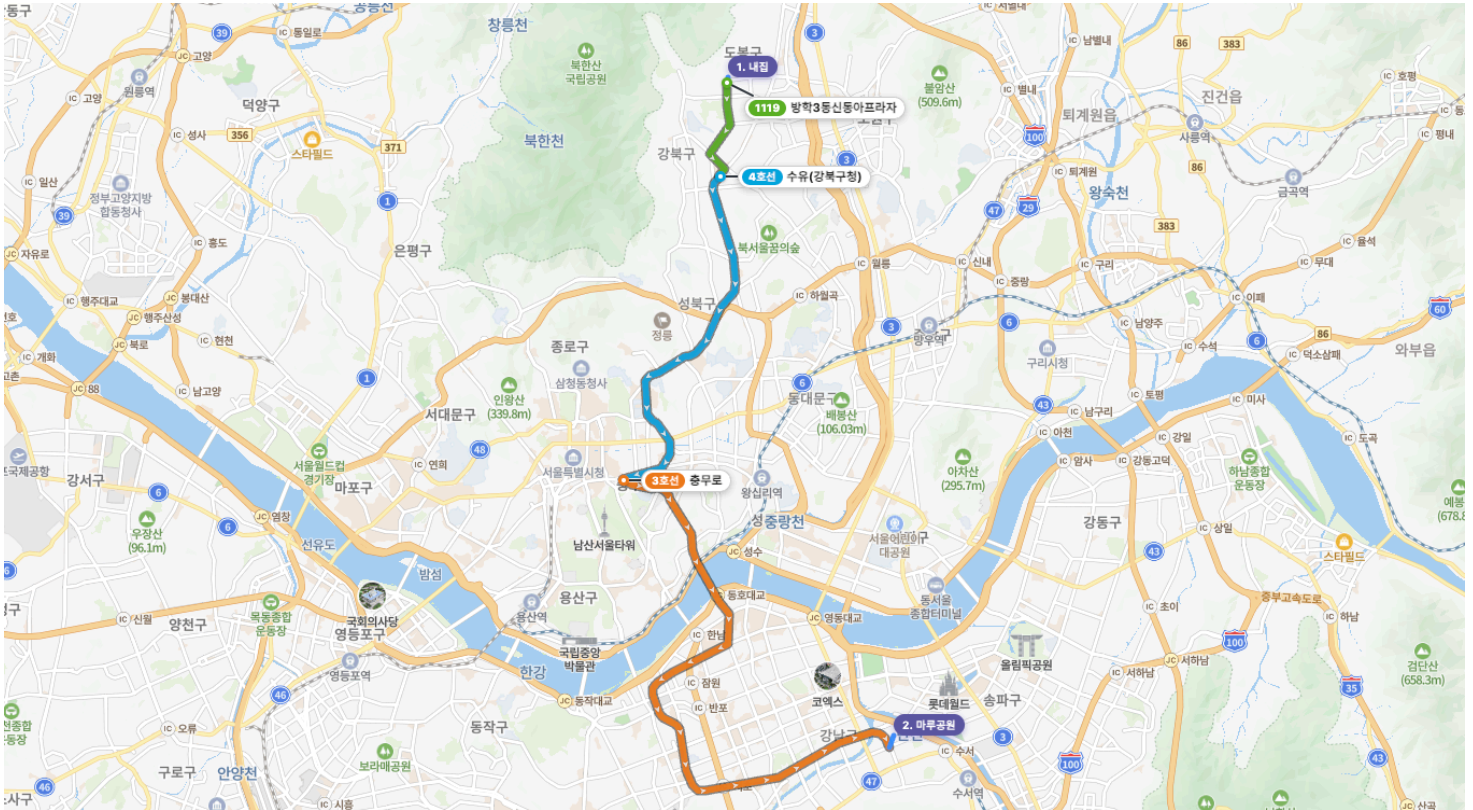
- **왜 성공만 캐시하나.** 빈 응답을 캐시하면 영구히 빈 채로 굳는다. 실패는 저장하지 않아 다음에 재시도하게 했다.
- **왜 in-memory인가.** transit_cache 의 route_key 컬럼은 이미 추가해 두어 DB 승격이 가능하다. 다만 단일 인스턴스·세션 단위로는 in-memory로 충분하고 회귀 위험이 작아 우선 채택했다. 다중 인스턴스로 확장하면 DB로 올린다.

4. 지도 폴리라인 시각화

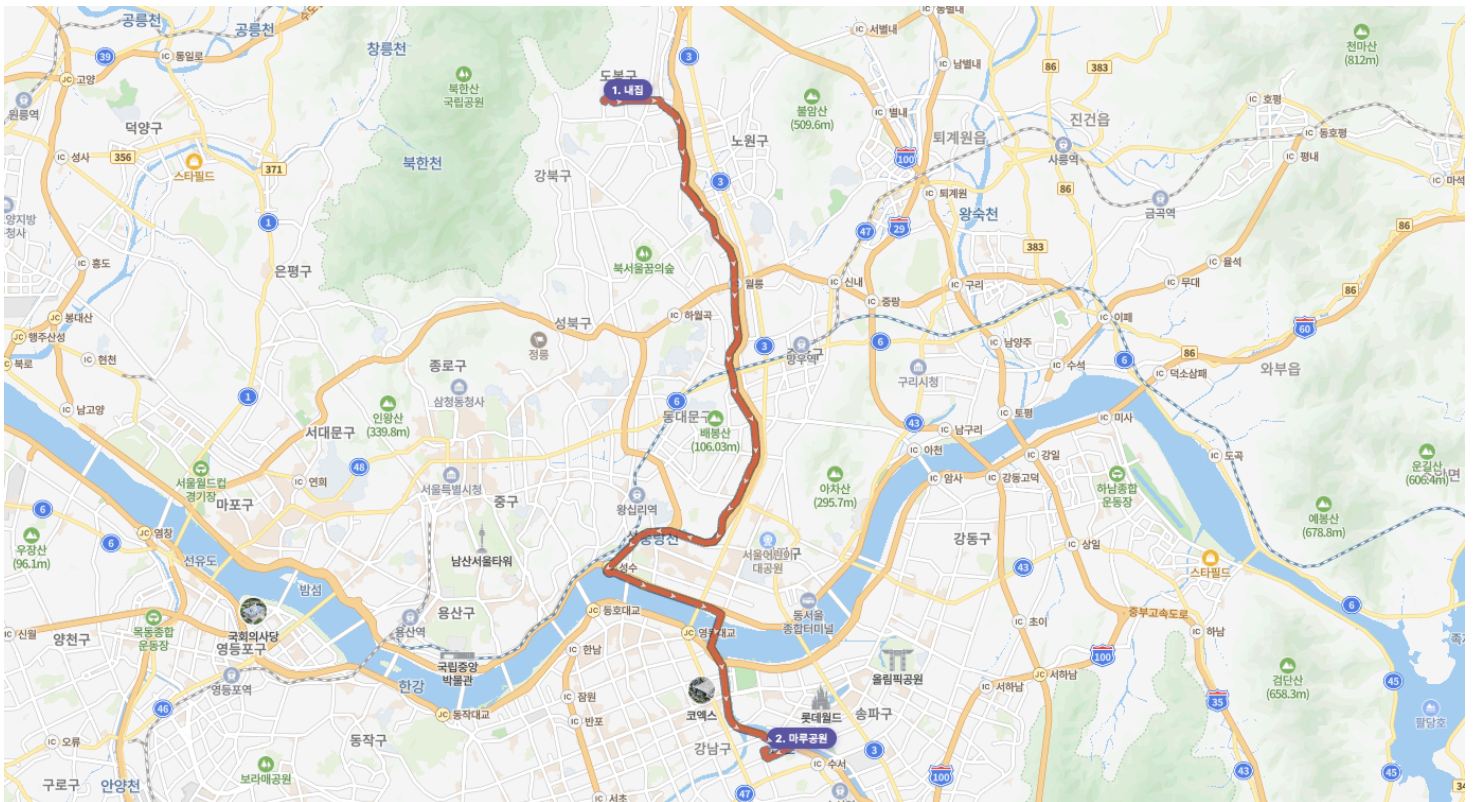
백엔드가 만든 좌표를 프론트가 Naver 지도에 그린다(drawDayRoute → drawPublicSegments).

- **좌표계 주의.** 저장은 [lng, lat] , Naver SDK는 LatLng(lat, lng) 순서다. 구간 좌표 seg.p 를 ([x, y]) => new naver.maps.LatLng(y, x) 로 뒤집어 매핑한다.
- **구간별 색.** segColor(seg) 가 구간 종류(seg.c)에 따라 색을 정한다 — 지하철은 호선 공식색(2호선 초록 등), 버스는 종류색, 열차(KTX)는 검정, 도보는 점선. 비-도보 구간은 검정 casing(9px) 위에 색선(6px)을 얹어 가독성을 높인다.
- **오버레이.** renderOverlays 가 폴리라인을 따라 방향 화살표를 ~30px 간격으로 회전 배치하고, 역/정류장 마커와 노선번호 라벨을 그린다. 라벨은 7단 세로 오프셋으로 **충돌 회피**하고, 리더 라인(흰 casing + 진한 선)으로 실제 정류장과 연결한다.
- **커스텀 장소 접근 구간 조립.** 커스텀(좌표 기반)은 관광지용 lane 캐시를 못 써 도시간 경로 선이 처음엔 KTX 역-역 직선만 그려졌다. → buildEnrichedRouteCoords 로 [출발지→역] + [본 구간] + [역→목적지] 를 이어붙여 한 줄로 연결했다.

- **알려진 한계.** 도시간 KTX 본 구간 자체는 loadLane 대상이 아니라 여전히 역-역 2점 직선(3단 폴백의 3번)이다. 접근 구간(버스·지하철)은 정류장 좌표로 선이 그려지고 모달 스텝에는 전부 표시된다. 철도 실폴리라인은 별도 과제.
- **프론트 캐시.** pillResults (key = from-to-hour → mode)에 패치 결과를 담아 같은 경로/모드/시간은 재호출하지 않는다. 모달 소요시간은 이 캐시를 읽는 computed로 API 없이 갱신된다.



대중교통 경로 — 구간별 색 폴리라인 + 환승/역 마커 + 방향 화살표



자동차 경로 — TMap 4옵선(추천/최단/무료/최소) 비교 + 도로 요약

② 어떻게 구현했나

- 토픽 설계
 - `/topic/trip/{id}` — 편집 이벤트(BLOCK_ADDED/MOVED/DELETED, CANDIDATE_ADDED/REMOVED, TRANSIT_RECALCULATED)
 - `/topic/trip/{id}/presence` — 커서/참가자 목록(PRESENCE_UPDATE)
 - `/app/trip/{id}/pointer` — 클라가 커서/grab/keepalive를 발행하는 목적지
- 인증(쿠키 기반, 헤더 토큰 없이)
 - 핸드셰이크: `JwtHandshakeInterceptor` 가 쿠키의 `access_token`을 검증해 세션 속성에 `memberId` 저장.
 - `JwtChannelInterceptor` : CONNECT 시 `memberId` 로 `Principal` 설정. SUBSCRIBE/SEND 시 해당 trip 접근 권한 검증.
 - 권한 캐시: 커서 전송은 매우 잦으므로 SEND마다 DB 조회하면 부담. 그래서 SUBSCRIBE/SEND 1회 검증 후 세션에 캐시. 단 협업자 제거/공유 변경 시 캐시가 낡지 않게 `TripAccessVersion` (일정별 세대 카운터)을 두고, 세대가 바뀌면 재검증한다.
- 편집 → 브로드캐스트 흐름
 - 편집은 REST(`placeBlock` / `updateBlock` / `removeBlock`)로 처리하고, 성공 시 `broadcast(tripId, TripEvent.of(...))` 로 변경을 알린다.
 - 수신측 `handleTripEvent` (`ScheduleBoard.vue`)가 이벤트 타입에 따라 `loadTrip()` (전체 재조회) 또는 `applyTransitUpdate()` (부분 패치) 수행.
 - 자기 이벤트 무시: 이벤트에 `actorId` 를 실어, 내가 일으킨 변경은 무시(내 화면은 이미 낙관적 UI로 반영됨).
 - 순서·중복 방어: `TripEvent.seq` (일정별 `AtomicLong` 단조 증가)를 broadcast 시 스탬프. 수신측은 `seq ≤` 마지막 처리 `seq` 면 무시 → 역전·중복 이벤트로 인한 불필요한 재조회 방지.
- 재연결: STOMP 자동재연결 대신 수동 재연결(`reconnectDelay:0` + 직접 `setTimeout`)로 제어. 재연결 성공 시 `onReconnect` → `loadTrip()` 로 끊긴 동안 놓친 변경을 한 번에 재동기화.
- 트랜잭션 위생: 이동시간 재계산(`recalculateTransitForDate`)은 외부 API(ODsay/TMap)를 호출하는데, 이를 편집 트랜잭션 커밋 이후 (`TransactionSynchronizationManager.afterCommit`)로 미뤄, 외부 지연 동안 DB 커넥션을 잡지 않게 했다.

③ 어려움

- 인증을 어디서 거는가: WebSocket은 일반 HTTP 필터 체인을 안 탄다. 핸드셰이크(쿠키)에서 인증하고, STOMP CONNECT/SUBSCRIBE/SEND 단계별로 권한을 나눠 거는 구조를 잡는 데 시행착오가 있었다.
- 전체 재조회 vs 부분 패치: 블록 이벤트는 단순화를 위해 `loadTrip()` 전체 재조회로 갔는데, 낙관적 UI(내가 방금 옮긴 블록)와 겹쳐 깜빡임이 생길 수 있다. 빈도·복잡도를 따져 전체 재조회를 유지하되, 이동시간만은 부분 패치(`applyTransitUpdate`)로 처리.
- 이벤트 유실/순서: 네트워크 순간 끊김 시 이벤트를 놓칠 수 있다. seq로 순서/중복은 방어하되, 유실분 재조회 전용 엔드포인트까지는 만들지 않고 "재연결 시 전체 재조회로 수렴"하는 단순 정책을 택했다(데이터 유실은 없음).

④ 고민·의사결정

- broadcast 시점: 현재 `BLOCK_*` broadcast는 트랜잭션 커밋 직전에 일어난다(수신측이 곧 재조회). 엄밀히는 "커밋 후 broadcast"가 더 정확하지만, 이벤트 순서·정합성 보강(seq 부여, afterCommit 재계산)을 우선하고 이 부분은 백로그로 남겼다.
- 부하 인식: presence broadcast는 인원이 늘면 트래픽이 $\sim O(n^2)$ 로 증가한다(인원마다 커서 전송 × 전원에게 재전송). 2~5인을 전제로 두되, 확장 시 좌표 델타 전송·coalescing이 1순위 보강임을 문서화했다.

2. 낙관적 락(Optimistic Lock) 기반 동시성 제어

① 왜 낙관적 락인가

- 문제: 처음엔 편집 경로가 `@Transactional` 단일 트랜잭션 + 락 없음이었다. 두 명이 같은 블록을 동시에 옮기면 나중 요청이 먼저 요청을 조용히 덮어쓴다(Last-Write-Wins → 데이터 유실).

- 선택지 비교

방식	막는 것	비용	판단
비관적 락(SELECT ... FOR UPDATE)	모든 경합	항상 락 점유, 드래그 UX와 안 맞음	과함
낙관적 락(version)	같은 row 동시 수정	컬럼 1개 + 조건부 UPDATE	채택
grab 서버 강제	드래그 중 블록 선제 차단	기존 인프라 호출만	보조
CRDT/OT(자동 병합)	모든 충돌 자동 해소	아키텍처 전면 전환(수 주)	ROI 낮음

- 낙관적 락이 맞는 이유: 우리 환경은 **충돌이 드물다**(2~5인 + 실시간 커서로 서로 인지). 비관적 락은 평소에도 손해지만 낙관적 락은 **평소 오버헤드 0, 실제 경합 때만 한 명이 거부된다**. 충돌 빈도가 낮을수록 유리.

② 어떻게 구현했나

- **version 컬럼**: trip_block.version INT NOT NULL DEFAULT 0 .
- **조건부 UPDATE**: updateWithVersion — UPDATE ... , version = version+1 WHERE id = ? AND version = ? → **affected rows 반환**. 0 행이면 그 사이 누가 먼저 바꾼 것 → 409 CONFLICT .
- **버전 왕복**: 클라가 블록을 읽을 때 받은 version (BlockItem)을 편집 요청(BlockUpdateRequest)에 동봉. 409를 받으면 토스트 + loadTrip() 으로 최신 상태 재조회(낙관적 UI 자가 복구).
- **두 층 방어**
 - grab(사전 차단): 누가 드래그 중인 블록을 다른 사람이 건드리면 getGrabOwner 검사로 즉시 409("ㅇㅇ님이 편집 중"). UX 친화적 선제 차단.
 - 낙관적 락(사후 안전망): grab을 안 거친 경로·grab이 풀린 순간까지 최종 정합성 보장.
- **충돌 기준 매트릭스**(이 설계의 핵심 — "무엇을 충돌로 볼 것인가")

시나리오	메커니즘	결과
같은 블록 동시 이동/리사이즈	version	둘째 409 → 재조회·재시도
드래그 중 블록을 타인이 수정	grab 게이트	즉시 409
서로 다른 블록을 각자 다른 위치로	독립 row·독립 version	충돌 없음 (정상 동시 처리)
transit 재계산 ↔ 사용자 편집	transit 전용 UPDATE(version 불변)	충돌 없음
삭제 vs 같은 블록 이동	이동의 versioned UPDATE가 0행	이동 측 409
다른 블록을 같은 시간대에 겹침	trip 행 FOR UPDATE + 겹침 검사	둘째 409

핵심 불변식: version 은 "같은 row"에만 작용 → 무관한 블록 편집·transit 재계산은 절대 오탐(false conflict)하지 않는다.

- **시간대 겹침 금지**(cross-row 제약): version으로는 못 막는다(다른 row). MySQL엔 배제 제약도 없어, 검사 직전 **trip 행을 SELECT ... FOR UPDATE 로 잠가** 같은 일정 편집을 직렬화했다. 둘이 동시에 겹치는 자리에 놓아도 둘째는 첫째 커밋 후 최신 상태로 검사 → 한쪽만 통과. 부수적으로 display_order MAX+1 race도 이 잠금 안에서 안전해졌다.
- **transit 분리**: 사용자 편집은 updateWithVersion (version+1), 시스템의 이동시간 재계산은 updateTransitById (transit 컬럼만, version·위치 불변)로 **물리적으로 다른 쿼리**로 나뉘었다. 이게 매트릭스 4행(오탐 없음)의 전제다.
- **권한 캐시 무효화**: 협업자 제거/역할·공유 변경 시 TripAccessVersion.bump(tripId) → STOMP 인터셉터가 다음 프레임에 권한 재검증.

③ 어려움 (실제로 부딪힌 버그들)

- **transit 재계산이 false-conflict를 유발**: 처음엔 재계산도 전체 UPDATE라 version을 올려, 사용자 편집 직후 시스템 재계산이 상대의 version을 무효화 → 멀쩡한 편집이 거부됐다. → transit 전용 UPDATE로 분리해 해결.
- **resize 후 재수정 시 단일 창에서도 409**(인상 깊었던 버그): onResizeEnd 는 성공 후 loadTrip 재조회를 안 해 로컬 ev.version 이 옛값 그대로 → 다음 수정에서 stale version → 0행 → 409. 협업도 아닌 한 창에서 재현됐다. → 성공 시 ev.version = ev.version + 1 로 서버의 version+1과 동기화해 해결.
- **grab 잠금이 안 풀림**: grab을 표시용으로만 쓸 땐 안 보였는데, 서버 게이트로 승격하니 드러났다. 드래그 종료 시 프론트가 targetBlockId:null 을 보내 서버 grabMap이 안 비워졌고, keepalive가 presence를 살려 stale evict도 안 돌아 **세션 내내 잠금 잔존** → 남이 그 블록을 영영 못 만짐. → 종료 시

"잡았던 블록 id"를 실어 명시 해제 + 본인 소유만 원자적 `remove(key,value)` .

- **removeCandidate TOCTOU**: "블록 있는지 검사 → 후보 삭제" 사이에 동시 배치가 끼면 FK(RESTRICT) 예외. → 삭제를 `DataIntegrityViolationException` 캐치로 감싸 동일 메시지(409)로 변환.

④ 고민·의사결정

- **자동 병합(CRDT/OT)을 왜 안 했나**: 블록을 op/CRDT로 재모델링하고 op 로그 영속·재동기화까지 가는 수 주 규모 전환인데, 우리 규모엔 과하다. 게다가 **CRDT/OT도 의미 충돌은 못 없앤다** — A는 9시, B는 14시로 같은 블록을 옮기면 둘 중 하나로 수렴할 뿐 "사용자가 원한 결과"를 보장하진 않는다(정합성만 보장). 그래서 "충돌 시 거부 → 재조화·재시도"가 더 단순·정직하다고 판단.
- **백로그를 기계적으로 구현하지 않음(중요)**: 1차 분석에서 "presence in-memory의 `computeIfAbsent` 복합 연산 비원자성"을 위협으로 적었지만, 재검증 결과 **`ConcurrentHashMap.computeIfAbsent`는 원자적**이라 오진이었다. 없는 버그를 막겠다고 커서 이동마다 도는 핫 패스에 락을 넣으면 성능만 깎인다 → 의도적으로 두고 분석을 정정했다. "백로그 항목마다 실제 위험인가·막는 비용이 합리적인가를 재검증"한 사례.
- **막을 가치 기준의 일관성**: "영속 데이터 유실 = 지금 막는다 / 표시·성능 문제 = 나중"으로 우선순위를 일관되게 잡았다. 그래서 같은 블록 충돌·시간 겹침은 하드 차단했고, broadcast 부하·깜빡임은 백로그로 미뤘다.
- **다인원 확장 인식**: 데이터 정합성은 인원이 늘어도 (낙관적 락+grab+권한세대로) 지켜지지만, 먼저 무너지는 건 **성능·UX**(409 거부율↑, broadcast ~O(n²), 재계산 중복)임을 명시했다.

3. 실시간 커서 트래킹 + 상대 좌표 동기화

① 왜 "상대 좌표 환산"이 필요한가

- **문제**: 처음엔 커서를 절대 픽셀(clientX/Y)로 주고받았다. 그러나
 - 사용자마다 모니터·창 크기가 다르고,
 - 같은 시간표라도 세로 스크롤 위치가 다르며(A는 1~8시, B는 9~15시 표시),
 - 지도 패널 폭·열림 여부도 다르다.
- 절대좌표를 그대로 공유하면 "5시 성심당 블록을 가리킨다"가 상대 화면에선 엉뚱한 곳에 찍힌다.
- **목표**: 커서를 "어느 영역(zone) + 그 zone의 의미 좌표"로 송수신해, 창·스크롤·폭이 달라도 모두가 "같은 의미의 지점"을 가리키게 한다. (픽셀 단위 완벽 일치는 비목표 — 의미 단위 정확이 목표)

② 어떻게 구현했나

- **좌표 모델**(전송 payload)
 - `zone: timetable | map | other`
 - `timetable`: `dayIndex` (어느 day 컬럼) + `colRatioX` (컬럼 내 가로 0~1) + `contentY` (시간축 콘텐츠 px)
 - `map`: `mapRatioX/mapRatioY` (컨테이너 내 비율 — lat/lng 정밀변환은 추후)
 - `grab`(드래그): `grabRatioX` (블록 폭 대비 잡은 가로) + `grabOffsetMin` (블록 top으로부터 분)
- **기준 DOM**: 세로는 `.day-cols` (시간 원점, 0~1440px 고정), 가로는 개별 `.day-col`. 두 요소의 `getBoundingClientRect()`가 스크롤을 이미 반영하므로 별도 `scrollTop` 보정이 필요 없다.
- **송신**(`buildPointerPayload`): 마우스 이벤트 → zone 판정 → `contentY = clientY - dayCols.top`, `colRatioX`, `dayIndex` 등으로 정규화.
- **수신**(`cursorStyle / ghostStyle / dropPreviewStyle`): payload를 자기 DOM 기준으로 역환산. 예) `top = 내 dayCols.top + contentY`, `left = 내 day-col.left + colRatioX × width`. 창·스크롤이 달라도 "같은 의미 위치"에 찍힘.
- **로직 분리**: 좌표 송수신을 `useCollabCursor.js` composable로 추출(2,000줄+ God Component `ScheduleBoard.vue`의 부담 분리).
- **표시 범위 제한**(두 종류, 서로 다른 문제)
 - **송신자 위치 기준**: 시간표 격자(`.day-cols`) 밖(헤더·시간축 거터·사이드바)은 `zone='other'`로 보내 숨김. 보드 밖으로 마우스가 나가면 `mouseleave`로 즉시 숨김.
 - **수신자 스크롤 기준**: 보내는 사람의 시간이 받는 사람의 시야 밖이면(스크롤 차이) 계산된 y가 시간표 위로 빠져나가 헤더에 그려진다 → 현재 보이는 격자 뷰포트(`gridViewport`) 밖이면 숨김.
- **presence 생명주기**
 - 화면에 있으면(idle여도) `keepalive heartbeat`(좌표/zone 미변경, lastSeen만 갱신)로 커서·아바타 유지.
 - 다른 창/탭으로 포커스를 잃으면(`blur / visibilitychange`) `leave` 전송 → `presence`에서 제거(아바타 원형까지 사라짐). 복귀 시 재등록.

- 창 종료는 WS onDisconnect 가 처리. 응답 없는 세션은 stale evict(기본 5초).
- **부드러움(적응형)**: 전송·보간 값을 부하에 따라 자동 조절.
 - 송신 throttle: AIMD — 움직임이 전송보다 빠르면(혼잡) 간격↑(back off), 한가하면 간격↓.
 - 수신 보간: CSS transition(--collab-smooth)을 **최근 수신 간격(dt)에 맞춤** → 한가하면 부드럽게(상한까지), 드래그로 촘촘하면 자동 스냅. transition이 항상 dt 이하라 **백로그가 구조적으로 불가능**.
 - 시연/실서비스 전환을 위해 throttle-smooth-keepalive 값을 .env (VITE_COLLAB_*)· config/collab.js 로 분리.

③ 어려움

- **세로축 보장**: day-col 콘텐츠 높이가 모두 동일(1440px)이라 contentY (콘텐츠 좌표)로 일반화하면 스크롤 무관해진다는 걸 잡는 게 핵심이었다. 처음엔 일반 커서가 이 보장을 전혀 안 해 어긋났다.
- **드래그 후 ghost가 한참 기어감**: onDragOver 가 매 이벤트마다 throttle 없이 커서를 보내 **백로그**가 쌓였다. 손을 떼도 큐가 천천히 소진되며 ghost가 늦게 따라옴. → 전송 throttle + (transition ≤ 수신 간격) 적응형으로 백로그 자체를 없앴.
- **idle 시 사라짐 vs 무관 영역 노출**: "가만히 있어도 유지"와 "영동한 곳에선 숨김"이 상충하는 듯 보였지만, **송신자 위치 기준 숨김**과 **포커스 기준 유지**는 조건이 달라 양립함을 정리.
- **스크롤 차이로 헤더 침범**: 위에서 설명한 수신자-스크롤 문제. 커서가 position:fixed 로 body에 teleport돼 클리핑이 없어 GNB/plan 헤더까지 올라가 그려졌다 → 가시 격자 범위 클램프로 해결.

④ 고민·의사결정

- **정확도 목표를 "의미 단위"로 못박음**: 픽셀 완벽 일치를 좇지 않고 "같은 day·시간·블록을 가리키면 OK"로 정의 → 구현 난이도와 효과의 균형.
- **지도는 비율 근사로 합의**: 지도 커서까지 lat/lng로 정밀 변환하면 줌·팬이 달라도 일치하지만, Naver projection 검증 비용이 커서 **비율 근사**로 두고 정밀화는 백로그(improvements.md)에 등재. 현재는 지도 zone 커서를 임시 숨김.
- **커서 가시화 = 동시성의 인적 안전장치**: 물리적 락은 아니지만 "지금 누가 어디를 만지는지"를 보여줘 **물리 충돌 빈도 자체를 낮추는 1차 완충**으로 봤다. 그래서 "다른 블록을 같은 슬롯에 놓는" 류의 드문 충돌은 하드 보장 대신 커서 인지 + 서버 정규화에 맡기는 판단이 가능했다(2장과 연결).
- **고정 튜닝 → 적응형**: 처음엔 throttle/smooth를 고정값으로 너프했는데, "리소스 여유 있으면 부드럽게, 쌓이면 즉시 낮춤"을 자동화하기 위해 AIMD·간격 연동 보간으로 전환. 수동 튜닝 없이 부하에 반응.